

LOREKEEPER

Contra as sombras da ignorância

Documentação técnica do sistema

Fatec Praia Grande

Análise e Desenvolvimento de Sistemas

Gabriel Xavier Oliveira

Murilo Furtado da Silva Neto

Paulo Gabriel Alves dos Santos

Orientador:
Bruno Baruffi

Bancada Avaliadora:

Resumo

Este documento descreve a especificação técnica do jogo digital educativo Lorekeeper: Contra as sombras da ignorância, uma experiência de RPG na web voltada a jogadores que desejam aprender outros idiomas por meio de combate por turnos, quizzes contextualizados e progressão em um hub de jogo. Apresentam-se objetivos, requisitos, arquitetura alvo (frontend em React, API REST em Spring Boot, PostgreSQL), mecânicas principais e diretrizes de implementação, testes e empacotamento (incluindo possível distribuição como aplicativo desktop via Electron).

Palavras-chave: jogo educativo; RPG; ensino de línguas; aplicação web; arquitetura em camadas.

Abstract

This technical document specifies Lorekeeper: Against the Shadows of Ignorance, a web-based educational RPG aimed at players interested in language learning through turn-based combat, in-game quizzes, and hub progression. It covers goals, functional and non-functional requirements, a layered target architecture (React SPA, Spring Boot REST API, PostgreSQL), core game mechanics, and notes on testing and packaging (including optional Electron).

Keywords: educational game; RPG; language learning; web application; layered architecture.

Sumário

Resumo.....	3
Abstract.....	3
1. Introdução.....	7
1.1 Contextualização.....	7
1.2 Fundamentação: jogos, vocabulário e aquisição de línguas.....	7
1.3 Problema e justificativa.....	9
1.4 Objetivos.....	9
2. Referencial teórico.....	10
2.1 Vocabulário em língua estrangeira e exposição ao input.....	10
2.2 Jogos digitais e aprendizagem de línguas (DGBL).....	10
2.3 Interatividade, imersão e ambientes virtuais.....	11
2.4 Multimodalidade, contexto e fixação lexical.....	11
2.5 O gênero RPG digital.....	12
2.6 Gamificação e distinção em relação ao jogo completo.....	12
2.7 Plataforma web e distribuição.....	12
3. Metodologia de desenvolvimento.....	13
3.1 Tipo de Desenvolvimento e Abordagem Geral.....	13
3.2 Como o Código é Escrito e Testado Junto (Fluxo de Desenvolvimento).....	13
3.3 Ferramentas e Ambientes Usados.....	14
3.4 Documentação e Registro de Decisões.....	14
3.5 Como Garantir Que o Código está Correto.....	15
3.6 Entrega e Operação do Jogo.....	15
4. Especificação do produto.....	16
4.1 Visão geral e proposta de valor.....	16
4.2 Escopo do produto e exclusões.....	16
4.3 Público-alvo, plataformas e contexto de uso.....	16
4.4 Casos de uso e fluxos principais.....	17
4.4.1 Módulo de Autenticação e Perfil de Usuário.....	17
4.4.2 Módulo de Gestão de Personagem, Inventário e Crafting.....	17
4.4.3 Módulo do Hub Central e Áreas Comuns.....	18
4.4.4 Módulo de Sistema de Batalha Educacional.....	19
4.4.5 Módulo de Missões, Conquistas e Progresso.....	19
4.5 Requisitos funcionais.....	20
4.6 Requisitos não funcionais.....	21
4.7 Regras de negócio, conteúdo pedagógico e premissas.....	22
5. Arquitetura do sistema.....	22
5.1 Visão lógica.....	22
5.2 Frontend.....	23

5.3 Backend.....	23
5.4 Dados.....	23
5.4.1 Diagrama do Banco de Dados.....	23
5.5 Integração cliente-servidor.....	26
5.6 Distribuição web e Electron.....	26
5.7 Integração com Backend Embarcado (Desktop).....	27
6. Mecânicas de jogo.....	28
6.1 Fluxo do jogador.....	28
6.2 Combate educativo.....	28
6.3 Progressão e metajogo.....	28
6.4 Idiomas e conteúdo.....	28
7. Empacotamento Desktop com Electron.....	28
7.1 Como Funciona Internamente (Arquitetura do Electron).....	29
7.2 O Que Acontece Quando o Jogo Abre (Fluxo de Inicialização).....	29
7.3 Otimizações para Rodar Rápido.....	30
7.4 O Servidor Incorporado (Backend Java).....	30
7.5 Como a Interface Conversa com o Sistema (Comunicação IPC).....	30
7.6 Como os Dados são Guardados (Armazenamento Local).....	31
7.7 Criando o Arquivo executável (Build e Empacotamento).....	31
7.8 Como os Desenvolvedores Testam Durante o Desenvolvimento.....	31
7.9 Entregando o Jogo Final (Distribuição).....	32
7.10 Protegendo o Computador do Jogador (Segurança).....	32
7.11 Encerrando o Jogo Corretamente (Shutdown).....	32
8. Como o Código está Organizado (Implementação).....	33
8.1 A Interface Visual (Frontend - React e TypeScript).....	33
8.2 O Motor do Jogo (Backend - Spring Boot).....	33
8.3 Um Exemplo Prático: Como Funciona uma Batalha.....	34
8.4 Interface Gráfica e Resultados Visuais.....	35
8.4.1 Módulo de Autenticação e Perfil.....	35
8.4.2 Exploração e Combate Educativo.....	36
9. Como Garantir Que o Jogo Funciona (Testes e Qualidade).....	38
9.1 Testando o Motor (Backend).....	38
9.2 Testando a Interface (Frontend).....	38
9.3 Testando Integração (Frontend + Backend).....	39
9.4 Testando a Versão Desktop (Electron).....	39
9.5 Testes Manuais.....	39
10. Conclusão.....	40
Referências.....	41
Apêndice A — Nota sobre arquitetura alvo vs. evolução do repositório.....	42

Apêndice B — Dicionário de Termos Técnicos.....	43
Apêndice C (Telas Adicionais).....	45
C.1 — Novo Jogo e Carregamento.....	45
C.2 — Configurações, Pausa e Status.....	46
C.3 — Criação e Seleção de Personagem.....	48
C.4 — Hub: Torre do Conhecimento (Pisos F1–F4).....	49
C.5 — Hub: Arena, Biblioteca, Loja e Estalagem.....	51

1. Introdução

1.1 Contextualização

A tecnologia permeia a vida cotidiana e os ambientes educacionais: em escolas e universidades é cada vez mais comum o uso de data shows, computadores, lousas digitais, slides, filmes, videoaulas e tecnologia móvel para viabilizar o ensino e a aprendizagem (SILVA; TOASSI, 2020). Dentro desse panorama, os videogames merecem atenção especial: após o surgimento, tiveram rápida evolução e passaram a fazer parte do cotidiano — inclusive Riley (2015), citado por Silva e Toassi (2020), aponta que 82% da população brasileira entre 13 e 52 anos joga algum tipo de jogo eletrônico.

Embora o entretenimento seja a finalidade principal na maior parte dos usos, há indícios de que jogos também podem contribuir para o processo de ensino e aprendizagem de línguas. Prensky (2007) sugere que o aluno se dispõe mais a aprender quando diversão e prazer fazem parte do processo; Gee (2007) destaca que o esforço para superar desafios no jogo ajuda o jogador a assimilar mensagens e conhecimento (SILVA; TOASSI, 2020). Estudantes que já são jogadores tendem a ter maior tempo de contato com a língua inglesa presente em interfaces e diálogos e, por exposição, familiarizam-se com vocabulário típico dos jogos.

O aprendizado de línguas estrangeiras (LE), por sua vez, exige input autêntico, uso ativo e fatores afetivos favoráveis — nem sempre presentes em materiais escolares convencionais. O gênero RPG (role-playing game) digital articula narrativa, metas de curto e longo prazo, feedback imediato e identificação com personagem, alinhando-se a desenhos que buscam motivação sustentada e repetição significativa de conteúdo linguístico. O projeto Lorekeeper: Contra as sombras da ignorância posiciona-se como RPG educativo na web para quem gosta de jogos e quer aprender ou reforçar outros idiomas em contexto lúdico.

1.2 Fundamentação: jogos, vocabulário e aquisição de línguas

Costa (2024), em O poder dos games: a influência na ampliação do vocabulário em língua estrangeira, trata da ampliação de vocabulário em LE como desafio comum a estudantes e aprendizes. Nesse contexto, os jogos surgem como proposta interessante, despertando interesse; contudo, há carência de conhecimento sobre o impacto preciso dos jogos na expansão do léxico. Torna-se relevante compreender em que medida características interativas e imersivas podem favorecer a assimilação de novas palavras e expressões.

Segundo Costa (2024), apesar da crescente adoção dos jogos como recurso complementar no ensino de LE, a maioria dos estudos concentra-se em métodos mais convencionais e menos envolventes; são escassas investigações aprofundadas sobre como interatividade, imersão e elementos narrativos impactam a expansão vocabular — lacuna reconhecida na literatura (Levay, 2015; Centenaro, 2016; Rocha, 2017; Silva, 2020; Leão-Junqueira, 2020; Murta, Pontes e Souza, 2022; Pushkina e Krivoslykova, 2022, conforme o levantamento do autor). O estudo de Costa (2024) combina questionários de campo e entrevistas semiestruturadas com alunos do 9º ano do ensino fundamental, do 3º ano do ensino médio e universitários em curso de inglês, buscando a percepção dos participantes sobre o efeito dos jogos no vocabulário em LE e sobre o papel da imersão e da interatividade.

No âmbito da justificativa, Costa (2024) observa que, em contexto de globalização, a proficiência em LE — em especial o inglês — é habilidade essencial, ao passo que ampliar o vocabulário fora da sala de aula tradicional permanece difícil para muitos estudantes (Franco, 2018, apud COSTA, 2024). Com a popularidade crescente dos jogos eletrônicos (Rocha, 2017, apud COSTA, 2024), abre-se espaço para explorar ambientes de imersão e interatividade que incentivem o contato contextualizado com o idioma, com potencial de levar a uma expansão lexical mais natural e significativa. Os resultados do trabalho indicam variedade de atitudes quanto aos games e ampliar vocabulário, mas a imersão em ambientes virtuais de LE foi amplamente reconhecida como eficaz; conclui-se que jogos podem oferecer aprendizado descontraído e enriquecimento do vocabulário, desafiando a visão de mero passatempo e apontando para abordagens pedagógicas inovadoras.

Silva e Toassi (2020), em artigo na Revista do GEL, investigam como videogames contribuem para a aprendizagem do inglês e como professores podem usar a ferramenta para motivar melhores resultados. O texto percorre origem e evolução dos jogos eletrônicos, define jogo (com base em Huizinga, 1993) e apresenta elementos listados por Prensky (2007) que tornam o videogame engajante — diversão, regras, objetivos, interatividade, retornos e aprendizagem, entre outros. Na classificação de Crawford (1982), destacam-se subgêneros como combate, aventura e RPG (Dungeons & Dragons), nos quais exploração, cooperação e conflito em ambientes ficcionais pedem participação ativa.

Para Crawford (2003), a interatividade é o grande diferencial dos jogos em relação a outras mídias: ciclo em que jogador e sistema (ou outro jogador) alternam escutar, pensar e agir — decisões disparam eventos que exigem ler ou ouvir instruções, o que favorece prática linguística, sobretudo em ambientes multijogador em que o inglês costuma funcionar como língua franca (SILVA; TOASSI, 2020). Gee (2005) reforça a relação imagem-palavra: novas palavras aprendem melhor quando ligadas a ações, imagens ou diálogos — o que ocorre, por exemplo, em RPGs com inventário nomeado e descrito. Os autores apresentam o jogo *Scribblenauts Unlimited* como alternativa de

aproximação ao inglês e discutem jogos eletrônicos alinhados a metodologias ativas, com protagonismo do aluno (Diesel; Baldez; Martins, 2017).

A pesquisa de Silva e Toassi (2020) aplica questionário a 46 pessoas sobre uso de jogos eletrônicos na aquisição do inglês; os dados indicam contribuição positiva, com destaque para ampliação de vocabulário e desenvolvimento da leitura. O artigo ressalta a relevância social e educacional da ferramenta e incentiva o professor a incluí-la em sala de aula.

Síntese para o Lorekeeper: (i) Costa (2024) problematiza a lacuna entre potencial dos games e evidências sobre interatividade, imersão e léxico, e mostra percepção favorável à imersão em LE; (ii) Silva e Toassi (2020) fundamentam motivação, interatividade, exposição lexical e papel do RPG, com dados empíricos sobre vocabulário e leitura; (iii) ambos legitimam desenhar um RPG web com desafios linguísticos explícitos (quiz) integrados ao combate e à narrativa. A documentação a seguir formaliza requisitos, arquitetura e mecânicas que materializam essa proposta.

1.3 Problema e justificativa

Permanecem desafios: materiais pouco alinhados a interesses de jogadores; necessidade de ferramentas que professores e autodidatas possam adotar; e, conforme Costa (2024), carência de estudos que detalhem como traços específicos dos jogos afetam a expansão vocabular — o que reforça a utilidade de produtos documentados, parametrizáveis e avaliáveis. Além disso, a expansão do léxico fora do arranjo escolar clássico continua difícil para muitos estudantes (Franco, 2018, apud COSTA, 2024), enquanto a popularidade dos jogos eletrônicos cria oportunidade de integrar lazer e aprendizagem (Rocha, 2017, apud COSTA, 2024).

Lorekeeper propõe unir o hábito de jogar a desafios linguísticos no combate e na exploração do hub, ampliando tempo de contato com o idioma-alvo e oferecendo feedback imediato, em linha com ambientes imersivos e interativos descritos por Costa (2024) e com os ganhos de vocabulário e leitura associados a jogos eletrônicos em Silva e Toassi (2020), além das perspectivas teóricas de engajamento e significado lexical (Prensky, 2007; Gee, 2005, 2007, conforme discutido por SILVA; TOASSI, 2020).

1.4 Objetivos

Objetivo geral:

Desenvolver e documentar um sistema de jogo RPG educativo na web que apoie o aprendizado de idiomas por meio de mecânicas de batalha, avaliação formativa (perguntas) e progressão estruturada em ambiente de hub.

Objetivos específicos:

- Especificar requisitos funcionais e não funcionais do Lorekeeper.
- Definir uma arquitetura em camadas, escalável e testável, para cliente web e API.
- Descrever o fluxo do jogador e as mecânicas que sustentam o conteúdo pedagógico.
- Registrar tecnologias, integração e possibilidades de distribuição (navegador e Electron).

2. Referencial teórico

Esta seção organiza conceitos que sustentam o desenho do Lorekeeper como RPG educativo voltado ao aprendizado de línguas, dialogando com trabalhos já discutidos na introdução (COSTA, 2024; SILVA; TOASSI, 2020) e com referências clássicas da área de jogos e educação.

2.1 Vocabulário em língua estrangeira e exposição ao input

A proficiência em língua estrangeira depende, entre outros fatores, de léxico receptivo e produtivo amplo — o que exige encontros repetidos e significativos com formas, sentidos e colocações. Fora do ambiente escolar formal, expandir o vocabulário costuma ser particularmente desafiador (Franco, 2018, apud COSTA, 2024). Daí a relevância de contextos que aumentem o tempo de exposição ao input autêntico sem depender exclusivamente de manuais ou aulas síncronas.

Atividades e mídias interativas podem funcionar como complemento: ao exigirem leitura de interface, diálogos e descrições contextualizadas, elevam a frequência de contato com o idioma-alvo. Silva e Toassi (2020) observam que estudantes que também são jogadores acabam mantendo maior tempo de contato com o inglês presente nos jogos e, por exposição, se familiarizam com vocabulário recorrente nesses ambientes.

2.2 Jogos digitais e aprendizagem de línguas (DGBL)

A aprendizagem baseada em jogos digitais (Digital Game-Based Learning, DGBL) investiga como mecânicas, narrativa e feedback de jogos podem apoiar objetivos educacionais. Não se trata apenas de “adicionar diversão”, mas de explorar ciclos de desafio, tentativa, erro e recompensa que mantêm o aprendiz engajado — ideia alinhada a Prensky (2007), para quem prazer e participação ativa favorecem a disposição para aprender, e a Gee (2007), segundo quem o esforço para superar obstáculos no jogo auxilia a assimilar mensagens e conteúdos (SILVA; TOASSI, 2020).

Costa (2024) problematiza que, embora os jogos apareçam como recurso complementar no ensino de línguas, grande parte da literatura ainda privilegia métodos convencionais; faltam investigações aprofundadas sobre como traços específicos dos

jogos — interatividade, imersão, narrativa — impactam a expansão vocabular. Esse recorte teórico justifica experiências como o Lorekeeper, nas quais esses traços são explícitos no desenho do software.

No plano quantitativo social, Riley (2015, apud SILVA; TOASSI, 2020) aponta elevada proporção de brasileiros que jogam jogos eletrônicos, o que reforça o potencial de alcance de um produto educativo nesse formato — especialmente quando acessível pela web ou por dispositivos móveis.

2.3 Interatividade, imersão e ambientes virtuais

Crawford (2003, apud SILVA; TOASSI, 2020) descreve a interatividade como processo cíclico em que dois agentes ativos alternam escutar, pensar e falar — no caso dos jogos, jogador e sistema (ou outro jogador). Cada decisão dispara eventos que exigem interpretação de texto ou de áudio, mantendo o ciclo de ação-feedback típico de ambientes de aprendizagem ativa.

A imersão em mundos ficcionais de língua estrangeira é central no debate sobre vocabulário: Costa (2024) relata que participantes de sua pesquisa reconheceram como eficaz a imersão em ambientes virtuais de LE nos jogos, em linha com a hipótese de que o engajamento com o cenário digital prolonga o contato contextualizado com o idioma.

Em jogos multijogador, a necessidade de coordenação verbal ou escrita entre jogadores de diferentes países frequentemente posiciona o inglês como língua franca — o que pode ampliar prática comunicativa espontânea (SILVA; TOASSI, 2020). Embora o Lorekeeper seja, em sua versão documentada, focado em fluxo single-player com servidor, essa literatura informa decisões de design (chat, missões cooperativas ou expansões futuras).

2.4 Multimodalidade, contexto e fixação lexical

Gee (2005, apud SILVA; TOASSI, 2020) argumenta que palavras novas são aprendidas com mais facilidade quando associadas a ações, imagens ou diálogos — isto é, quando o significado é ancorado em experiência multimodal e situada. RPGs com inventário, descrições de itens e missões textuais exemplificam esse acoplamento entre léxico, visual e objetivo de jogo.

No Lorekeeper, perguntas e feedback visual no combate reforçam o mesmo princípio: o item linguístico não aparece isolado, mas vinculado a consequências na narrativa (acerto ou erro no combate), aproximando-se de uma abordagem semântica e contextual em vez de lista memorizada.

Silva e Toassi (2020), com dados de questionário aplicado a 46 pessoas, associam uso de jogos eletrônicos a ganhos relatados em vocabulário e em leitura no processo de aquisição do inglês — resultado compatível com a hipótese de exposição frequente e contextualizada.

2.5 O gênero RPG digital

Em taxonomias de jogos digitais, o RPG (role-playing game) costuma envolver exploração, progressão de personagem, narrativa e, em muitos casos, cooperação ou conflito em ambientes ficcionais (Crawford, 1982, apud SILVA; TOASSI, 2020). Essa estrutura permite distribuir objetivos de aprendizagem ao longo de zonas, missões e encontros com inimigos — cada “boss” ou evento pode carregar um conjunto de desafios linguísticos sem quebrar a coerência diegética.

Silva e Toassi (2020) apresentam o jogo Scribblenauts Unlimited como exemplo de aproximação ao inglês por meio de resolução de problemas e vocabulário ativo. De forma análoga, o Lorekeeper combina exploração de hub, combate por turnos e questionários explícitos, explicitando o componente pedagógico sem abandonar a fantasia narrativa.

2.6 Gamificação e distinção em relação ao jogo completo

Gamificação designa o uso de elementos típicos de jogos (pontos, níveis, rankings, conquistas) em contextos não lúdicos ou para reforçar comportamentos (Deterding et al., 2011). No Lorekeeper, há elementos compatíveis com gamificação — XP, nível, achievements —, porém o objeto de estudo é um jogo digital completo, com regras, narrativa e estética próprias, e não apenas a camada de recompensas aplicada a uma atividade convencional.

Essa distinção é relevante para o referencial: o engajamento provém do papel assumido pelo jogador, do conflito narrativo e do feedback imediato do sistema de combate, alinhados ao que Prensky (2007) descreve como características estruturais dos videogames (objetivos, interatividade, retornos que propiciam aprendizagem) conforme sistematizado por Silva e Toassi (2020).

2.7 Plataforma web e distribuição

A entrega como aplicação web reduz barreiras de instalação e facilita atualização de conteúdo linguístico e de regras de jogo no servidor. O empacotamento opcional com Electron permite distribuir o mesmo cliente como aplicativo desktop (atalho, ícone, políticas de atualização) sem duplicar a lógica de negócio, mantendo a arquitetura alinhada às práticas atuais de desenvolvimento de produto educacional digital.

3. Metodologia de desenvolvimento

Esta seção descreve como o produto Lorekeeper foi (e continua sendo) conduzido do ponto de vista de engenharia de software: decisões de processo, ferramentas, ambientes, rastreamento de requisitos e práticas de qualidade. Complementa o referencial teórico (fundação pedagógica e de jogos) com a dimensão metodológica própria do desenvolvimento de sistemas.

3.1 Tipo de Desenvolvimento e Abordagem Geral

O Lorekeeper é um projeto de desenvolvimento de software prático e aplicado: partimos de um problema real (como ensinar idiomas através de jogos) e criamos uma solução de software funcional e bem documentada. O foco é entregar um produto que funciona, com especificação clara do que deveria fazer e testes para verificar se faz.

O desenvolvimento não segue um único plano gigante, mas sim um processo em ciclos: construímos um pouco de funcionalidade (ex: a tela de login), testamos com o servidor, corrigimos problemas, e depois passamos para a próxima funcionalidade (tela do hub). Esse jeito permite detectar problemas cedo, quando são mais fáceis de corrigir. Se descobrirmos que a forma como a interface fala com o servidor tem um erro, arrumamos logo, não no final.

No futuro, esse produto pode ser testado com jogadores reais para avaliar se realmente ajuda no aprendizado de idiomas — mas esse teste não faz parte deste documento técnico, é para trabalhos posteriores.

3.2 Como o Código é Escrito e Testado Junto (Fluxo de Desenvolvimento)

A interface visual (frontend) e o servidor (backend) trabalham separadamente, mas de forma coordenada. A interface sabe o que pedir ao servidor, e o servidor sabe que tipo de resposta deve dar. Antes de começar a programar, os desenvolvedores combinam esse 'contrato' (que dados serão trocados, em que formato).

O fluxo é assim: (1) Combinam qual será a mensagem que a interface manda para o servidor; (2) Um programador implementa a lógica no servidor (o que fazer com essa mensagem); (3) Outro programador faz a interface usar o resultado; (4) Testam para ver se funcionou. Se não funcionar, descobrem o problema e corrigem logo.

Os códigos ficam em dois projetos separados (um para a interface visual, outro para o servidor), mas sincronizados em um único lugar (repositório). Isso permite que dois programadores trabalhem ao mesmo tempo: um mexendo na interface e outro no servidor, sem um atrapalhar o outro.

Para que todos os desenvolvedores e avaliadores trabalhem com exatamente os mesmos dados, usamos um container (uma espécie de 'computador virtual' dentro do computador) que já vem pronto com o banco de dados configurado. Também documentamos todas as mudanças no banco de dados de forma versionada, como quem fez, quando e por quê.

3.3 Ferramentas e Ambientes Usados

Cada parte do projeto (interface visual e servidor) usa ferramentas específicas que facilitam o desenvolvimento. Todas essas ferramentas são gratuitas e bem conhecidas na indústria:

Para a Interface Visual (Frontend):

- React: Biblioteca que facilita criar interfaces visuais interativas.
- TypeScript: Versão mais segura de JavaScript que avisa se houver erros antes de executar.
- Vite: Ferramenta que faz o código ficar rápido e pronto para usar.
- Styled-components: Forma de desenhar a aparência dos elementos (cores, tamanhos, animações).
- Framer Motion: Biblioteca que adiciona animações fluidas de entrada e saída nos elementos da interface, como transições entre telas e cartas de monstros.

Para o Servidor (Backend):

- Java 17: Linguagem de programação robusta e confiável.
- Spring Boot: Framework que facilita criar um servidor Java rapidamente.
- JWT (tokens de segurança): Mecanismo que permite o jogador ficar registrado sem precisar enviar senha toda vez.
- PostgreSQL: Banco de dados que armazena informações dos jogadores, personagens e saves.

Para o Desenvolvimento Integrado:

- Docker: Permite que o servidor e banco de dados rodem em um 'computador virtual' isolado, garantindo que funciona igual em qualquer máquina.
- Git: Controla todas as mudanças no código (quem fez, quando, por quê), permitindo que vários programadores trabalhem juntos.
- GitHub/GitLab: Lugar na internet onde o código fica guardado e seguro.

3.4 Documentação e Registro de Decisões

Tudo que o jogo deveria fazer (requisitos — ver Seção 4) é documentado antes de começar a programar. Assim, quando o código está pronto, podemos verificar se realmente fez o que foi prometido.

Os nomes das funcionalidades no código correspondem aos nomes nos requisitos, facilitando encontrar e rastrear: 'a autenticação do jogador' está no requisito RF01 e no código em `auth/`.

Quando há mudanças importantes no projeto (ex: 'vamos adicionar multiplayer', 'vamos mudar de banco de dados'), documentamos qual foi a decisão, por quê, e qual melhoria traz ao projeto através de documentos `Readme`.

3.5 Como Garantir Que o Código está Correto

Qualidade é garantida em várias camadas, como explicado na Seção 9. Além dos testes automatizados, usamos outras práticas:

- Verificação automática de estilo: Um programa avisa se o código tem erros óbvios ou não segue padrões da equipe.
- Verificação de tipos: Antes até de rodar o código, o TypeScript avisa se estamos usando um tipo de dado errado (ex: tentar fazer conta com texto).
- Revisão por pares: Outro programador lê o código antes dele entrar no projeto, sugerindo melhorias ou apontando bugs.
- Testes em cenários reais: Como o jogo tem muita interação visual, a gente testa manualmente as transições entre telas, os efeitos visuais e como o jogo responde a ações do jogador.

3.6 Entrega e Operação do Jogo

O Lorekeeper pode ser entregue de dois jeitos: (1) Como um jogo web — o jogador acessa por um link no navegador; (2) Como um aplicativo desktop — o jogador baixa um `.exe` e instala no computador. Em ambos os casos, o processo de empacotamento é automatizado.

- Processo automático: Um script compila o código (junta, otimiza), testa, e gera o arquivo final (seja um site pronto ou um `.exe`).
- Segurança: Senhas, chaves e URLs do servidor não ficam no código (que é público). Em vez disso, ficam como variáveis de ambiente que mudam conforme o lugar onde o jogo roda (teste, demonstração, produção).
- Monitoramento: Quando o jogo está operando, ficamos de olho em erros que acontecem e fazemos backups regulares do banco de dados.

4. Especificação do produto

Esta seção consolida a definição do produto sob a ótica de engenharia de requisitos: o que o Lorekeeper é, para quem se destina, o que está dentro e fora do escopo, quais fluxos o usuário percorre e quais condições de qualidade devem ser atendidas. Os identificadores RFxx e RNFxx apoiam rastreabilidade com a implementação (Seções 5 a 7) e com eventuais matrizes de teste.

4.1 Visão geral e proposta de valor

Lorekeeper: Contra as sombras da ignorância é um RPG educativo na web em que a narrativa fantástica personifica a ignorância e distrações como forças a serem enfrentadas. O jogador assume um personagem com classe e atributos, explora um hub composto por zonas temáticas (por exemplo torre de estudos, arena, biblioteca, comércio e pontos de descanso) e entra em combates por turnos contra criaturas que metaforizam equívocos linguísticos ou hábitos de estudo frágeis.

A proposta de valor combina entretenimento de gênero RPG com prática deliberada de idioma: ações relevantes no combate ou na progressão exigem acerto em desafios linguísticos (perguntas de múltipla escolha, associações ou formatos evolutivos armazenados no servidor), com feedback imediato. Diferente de um aplicativo apenas gamificado (lista de tarefas com pontos), o software oferece mundo coerente, personagens, economia simplificada (ouro, itens) e metas de longo prazo (nível, conquistas), sustentando engajamento repetido.

O produto é entregue prioritariamente como aplicação web responsiva; a mesma base pode ser empacotada com Electron para desktop, preservando a API REST como única fonte de regras de negócio e persistência.

4.2 Escopo do produto e exclusões

Dentro do escopo encontram-se: cadastro e login; gestão de perfil de jogo; criação e recuperação de personagem; navegação pelo hub e acionamento de zonas interativas; fluxo completo de batalha com sincronização de estado via API; banco de perguntas e metadados linguísticos; salvamento em slots; tutorial e diálogos scriptados; loja/progressão onde previstos no repositório; sistema de conquistas; e infraestrutura mínima de segurança (JWT, validação, CORS em produção).

4.3 Público-alvo, plataformas e contexto de uso

O público principal inclui adolescentes e adultos que já jogam RPGs ou jogos casuais com narrativa e que desejam aprender ou reforçar um segundo idioma fora do formato tradicional de livro + lista de vocabulário. O sistema também se presta a autodidatas

motivados e a estudantes que buscam complemento lúdico, sem substituir planejamento pedagógico formal de instituições — salvo uso supervisionado pelo professor.

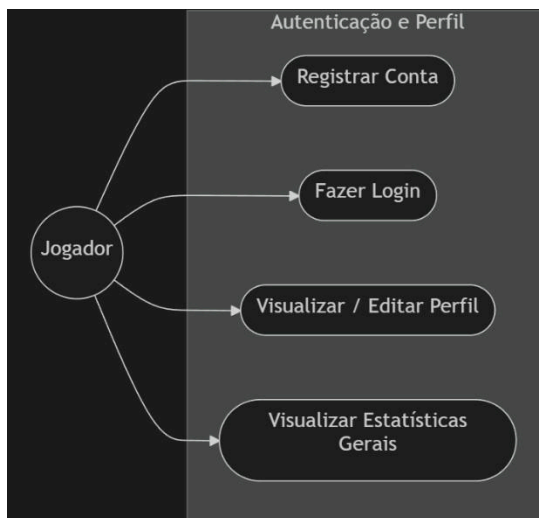
Plataformas-alvo: navegadores modernos (Chrome, Edge, Firefox, Safari recentes) em desktop e, de forma responsiva, telas médias; opcionalmente janela Electron em Windows (e demais SO suportados pelo empacotador). O uso pressupõe conexão com a internet para autenticação e sincronização com a API, salvo cenários locais de desenvolvimento.

Sessões típicas podem variar de dez minutos (um combate e revisão) a mais de uma hora (progressão no hub e exploração). A interface deve tolerar interrupções: salvamento disponível conforme RF06 reduz perda de progresso.

4.4 Casos de uso e fluxos principais

Esta seção detalha os atores, os casos de uso de alto nível e as interações do jogador com o sistema, divididos em módulos funcionais.

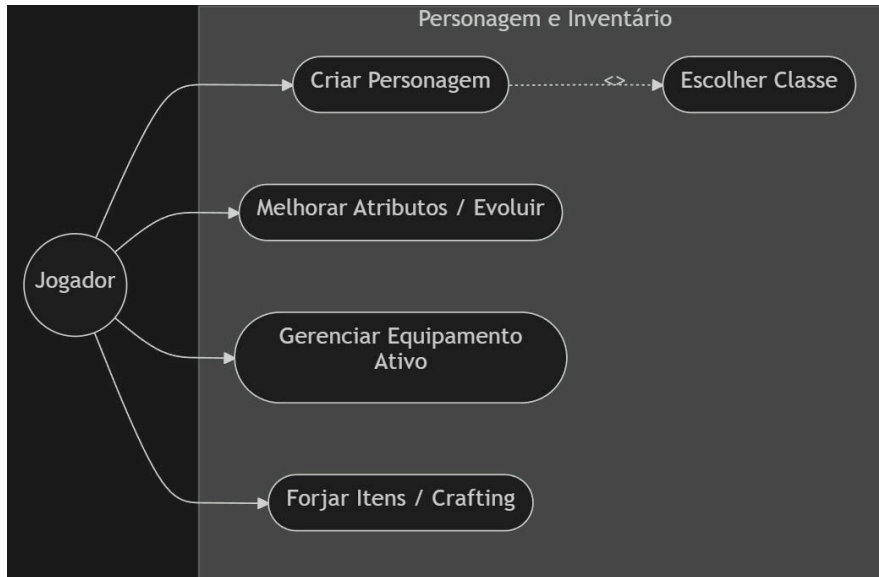
4.4.1 Módulo de Autenticação e Perfil de Usuário



Atores: Jogador

Fluxo: O jogador registra-se, realiza login, gerencia seu perfil e consulta suas estatísticas.

4.4.2 Módulo de Gestão de Personagem, Inventário e Crafting



Atores: Jogador

Fluxo: Criação e evolução de personagem, manipulação de inventário de equipamentos ativos e forja (crafting) de novos itens.

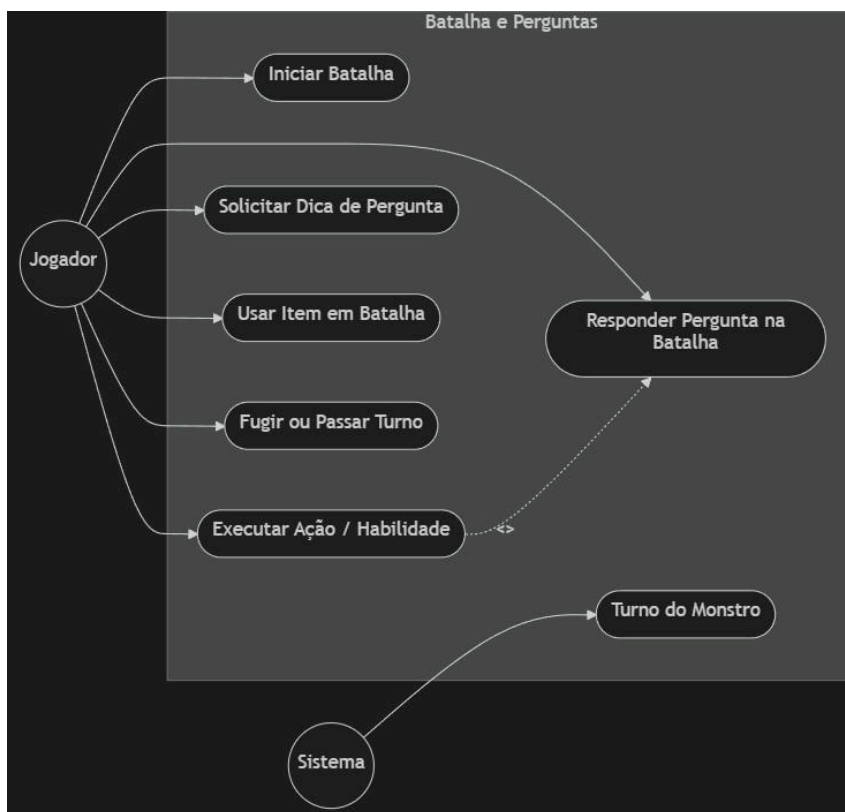
4.4.3 Módulo do Hub Central e Áreas Comuns



Atores: Jogador

Fluxo: Exploração pacífica do mundo do jogo, incluindo Loja, Torre (para cura e habilidades), Biblioteca para leitura, diálogos com NPCs no palco e salvamento de progresso.

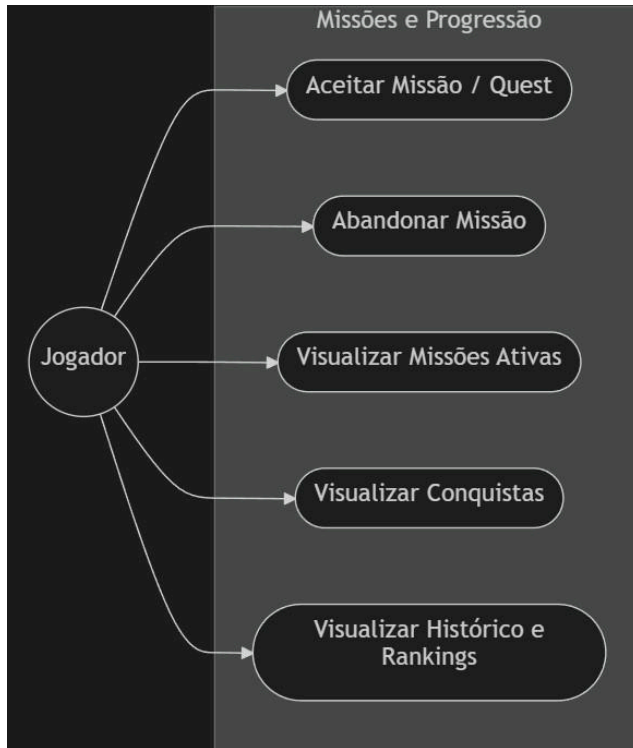
4.4.4 Módulo de Sistema de Batalha Educacional



Atores: Jogador, Sistema

Fluxo: O jogador inicia a batalha e toma decisões táticas que são entrelaçadas com perguntas e conteúdo linguístico a serem respondidas para efetivar as ações. O sistema gerencia o turno do monstro.

4.4.5 Módulo de Missões, Conquistas e Progresso



Atores: Jogador

Fluxo: Aceitação e abandono de missões da Torre, além do acompanhamento de conquistas desbloqueadas, históricos de batalha e rankings.

Fluxo feliz resumido: autenticação → hub → escolha de atividade (batalha, loja, missão) → resolução de desafios linguísticos embutidos → persistência de estado → loop até objetivos do jogador ou fim de sessão.

4.5 Requisitos funcionais

Os requisitos funcionais descrevem comportamentos observáveis do sistema. A numeração é estável para rastreo; novos requisitos devem receber identificadores novos (RF11, RF12, ...) em revisões futuras.

- RF01 — Cadastro e autenticação de usuário com sessão segura (JWT) e encerramento de sessão (logout) coerente no cliente.
- RF02 — Criação, listagem e seleção de personagem com classe e atributos distintos persistidos no servidor.
- RF03 — Navegação por hub com zonas reconhecíveis (ex.: torre, arena, biblioteca, loja, estalagem, mural de missões) e transição de interface sem perda de estado autenticado.
- RF04 — Início, andamento e resolução de batalha por turnos com estado autoritativo no servidor e atualização consistente no cliente.

- RF05 — Apresentação de perguntas (quiz) vinculadas a ações de combate ou eventos, com registro de resposta e feedback ao jogador.
- RF06 — Sistema de salvamento em múltiplos slots, com criação, sobrescrita e carregamento de progresso validado pelo backend.
- RF07 — Tutorial e diálogos orientativos na primeira experiência ou quando reativados, sem bloquear indefinidamente o jogo livre.
- RF08 — Loja, inventário e progressão (XP, nível, ouro, recompensas) calculados conforme regras centralizadas no servidor.
- RF09 — Conquistas (achievements) disparadas por eventos de jogo, persistidas e exibidas ao jogador.
- RF10 — Conteúdo linguístico parametrizável por idioma, nível, tema ou tags, permitindo evolução do banco de perguntas sem alterar o executável do cliente.
- RF11 — Tratamento de erros de rede e de API: mensagens compreensíveis ao usuário em falhas de login, timeout ou respostas 4xx/5xx recorrentes.
- RF12 — Missões ou objetivos secundários no hub (quando implementados) com estado rastreável (aceite, progresso, conclusão).
- RF13 — Perfil mínimo de usuário: alteração de dados permitida conforme política de segurança (senha, e-mail, onde aplicável).
- RF14 — Acessibilidade básica: contraste e tamanhos compatíveis com leitura em diferentes resoluções; foco navegável em formulários críticos (login/registro).
- RF15 — Internacionalização da interface do aplicativo (menus e rótulos) quando distinta do idioma-alvo pedagógico — configurável ou fixada em português na versão documentada.

4.6 Requisitos não funcionais

Os requisitos não funcionais fixam qualidades do sistema: desempenho, segurança, manutenção, operabilidade e portabilidade. Métricas quantitativas exatas (por exemplo, latência p99) podem ser negociadas em ambiente de produção e registradas em SLA interno.

- RNF01 — Interface responsiva para navegadores modernos; layout utilizável em larguras típicas de notebook e monitores widescreen.
- RNF02 — Comunicação com API REST; uso de HTTPS em produção; sem exposição de segredos ou tokens em repositório público.
- RNF03 — Tempo de resposta perceptível: ações de combate e carregamento de perguntas devem completar em ordem de poucos segundos em rede estável, evitando travamentos da UI sem indicador de carregamento.
- RNF04 — Código modular no backend (camadas ou pacotes por contexto) e no frontend (features/hooks), facilitando manutenção e testes.

- RNF05 — Migrações de esquema de banco versionadas (ex.: Flyway), com histórico reproduzível entre ambientes.
- RNF06 — Possibilidade de empacotar o cliente web com Electron sem duplicar regras de negócio; mesma API para navegador e desktop empacotado.
- RNF07 — Segurança: senhas armazenadas com hash forte (ex.: BCrypt); validação de entrada nos endpoints; proteção contra uso indevido de tokens expirados ou inválidos.
- RNF08 — Disponibilidade: backend e banco configuráveis para reinício automático em ambiente containerizado; logs estruturados ou rastreáveis para diagnóstico.
- RNF09 — Privacidade: dados pessoais mínimos necessários ao jogo; alinhamento a boas práticas de LGPD em implantações que colem dados de titulares no Brasil.
- RNF10 — Compatibilidade: versões de Java e Node alinhadas aos arquivos de projeto (`pom.xml`, `package.json`); documentação de variáveis de ambiente obrigatórias.

4.7 Regras de negócio, conteúdo pedagógico e premissas

Regras de negócio centrais: (i) o servidor é fonte da verdade para HP, ouro, inventário, resultado de perguntas e progressão — o cliente não deve poder alterar esses valores sem validação; (ii) uma resposta incorreta em quiz pode implicar penalidade narrativa no combate (por exemplo falha de ataque ou dano recebido), conforme implementação da engine de batalha; (iii) conteúdo linguístico deve ser consistente com o idioma-alvo selecionado ou com o contexto da pergunta, evitando mistura não intencional de línguas na mesma avaliação, salvo exercícios explicitamente contrastivos.

Premissas: existe instância de API acessível e banco PostgreSQL operacional; o jogador possui hardware e rede mínimos; para Electron empacotado, runtime Java disponível quando o produto incluir inicialização local do JAR. Dependências externas incluem provedor de hospedagem (se deploy remoto), certificados TLS e política de backup do banco.

5. Arquitetura do sistema

Foi utilizada uma arquitetura em camadas com fronteiras explícitas, aderente ao que o projeto implementa em termos de stack, padronizada em pastas e responsabilidades:

5.1 Visão lógica

- Camada de apresentação: SPA React (TypeScript), roteamento por estado de jogo e componentes por feature (auth, hub, battle, quiz, settings).
- Camada de aplicação (API): controladores REST finos, serviços com regras de negócio, validação de entrada.

- Camada de domínio: entidades, enums e políticas de combate/perguntas.
- Camada de infraestrutura: JPA/repositórios, Redis para cache, migrações SQL.

5.2 Frontend

Cliente único (Vite + React): Context API para autenticação, jogo, tutorial e tela cheia; serviços HTTP isolados; hooks por fluxo (batalha, save, hub); telas organizadas por funcionalidade em features/. Estado global centralizado via GameContext; navegação por máquina de estados; dados de servidor como fonte da verdade para HP, ouro, inventário e resultado de batalhas.

5.3 Backend

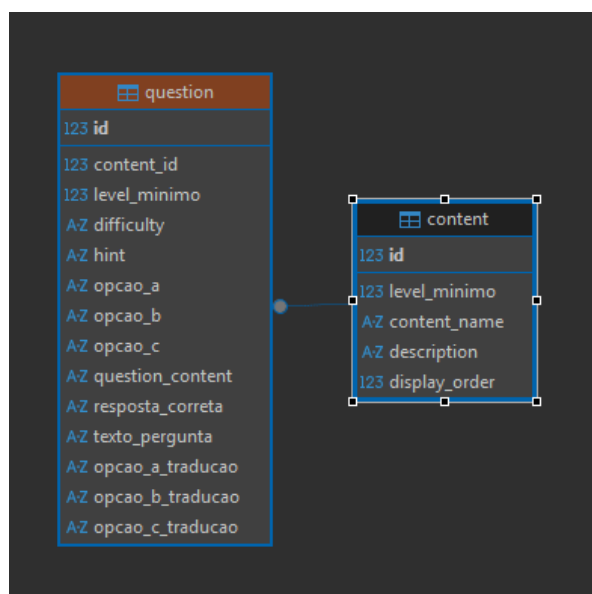
Spring Boot 3.x (Java 17+): módulos por bounded context em pacotes — auth, user, character, battle, question, hub, save, achievement, dialog, tutorial. Segurança com Spring Security + JWT. DTOs de entrada/saída explícitos para evitar vazamento de entidades JPA.

5.4 Dados

PostgreSQL como banco relacional principal; Flyway para evolução de esquema; Redis para cache de leitura (perguntas, catálogos).

5.4.1 Diagrama do Banco de Dados

A estrutura das tabelas do banco de dados segue um modelo entidade-relacionamento (ER), organizando as informações do jogo de forma lógica e eficiente. O diagrama abaixo mostra como as entidades (tabelas) se relacionam entre si. Seguem algumas relações de tabelas:







5.5 Integração cliente-servidor

Contratos REST JSON; tratamento de erros padronizados; variáveis de ambiente para URL da API no front; CORS configurado para origens autorizadas.

5.6 Distribuição web e Electron

O build de produção gera assets estáticos servidos por CDN ou servidor estático; um wrapper Electron carrega a mesma URL ou arquivos locais e expõe janela nativa, ícone e atualizações opcionais, sem duplicar lógica de negócio no desktop.

5.7 Integração com Backend Embarcado (Desktop)

Na distribuição desktop via Electron, o JAR do backend é embutido em resources e executado automaticamente quando a aplicação é iniciada. O processo principal (main.cjs) gerencia: (i) limpeza de porta em caso de processos prévios; (ii) inicialização inteligente do JAR (bootup rápido em execuções posteriores); (iii) comunicação entre renderer e main via IPC; (iv) armazenamento local de banco SQLite em userData. A API rodando localmente elimina dependência de rede externa, permitindo jogo offline após a primeira sincronização.

6. Mecânicas de jogo

6.1 Fluxo do jogador

- Autenticação → menu principal (novo jogo / carregar save).
- Seleção de classe e confirmação de tutorial.
- Carregamento e entrada no hub — navegação entre zonas temáticas.
- Encontro com inimigo → batalha por turnos; ações exigem acerto em desafios linguísticos.
- Quiz modal ou tela dedicada conforme estado (pergunta atual, alternativas, feedback).
- Recompensas, level up, loja, diálogos e retorno ao hub.

6.2 Combate educativo

O combate combina mecânicas clássicas (HP, turnos, habilidades de monstro e jogador) com gatilhos pedagógicos: acertos em perguntas liberam ou potencializam ataques; erros podem representar falhas de 'precisão linguística' narrativamente integradas.

6.3 Progressão e metajogo

Experiência, ouro, itens, conquistas e scripts de tutorial estruturam o loop de longo prazo. A biblioteca e outras zonas do hub podem concentrar conteúdo de estudo ou revisão.

6.4 Idiomas e conteúdo

O banco e a API suportam bancos de perguntas com metadados (idioma, dificuldade, tags). A narrativa reforça o tema do conhecimento versus distração/ignorância, alinhando tom fantástico ao objetivo educacional.

7. Empacotamento Desktop com Electron

O Lorekeeper pode ser entregue de duas formas: como um jogo acessível pelo navegador da web (como qualquer site) ou como um aplicativo desktop baixável (arquivo .exe para Windows). Para criar essa versão desktop, utilizamos uma tecnologia chamada Electron, que funciona como um 'embalador' para transformar a aplicação web em um programa tradicional do computador.

Pense no Electron como um embrulho que pega três componentes principais: (i) a interface visual do jogo (React), que é a mesma do navegador; (ii) um gerenciador de

operações do sistema (que cria a janela, controla menus e acesso a arquivos); (iii) o servidor do jogo (backend em Java), que roda diretamente no computador do usuário, sem precisar de internet para jogar.

7.1 Como Funciona Internamente (Arquitetura do Electron)

Para proteger a segurança e funcionalidade, o Electron divide o aplicativo em três partes independentes que conversam entre si. Pense como em um prédio com departamentos que precisam se comunicar:

- Departamento Principal (Main Process): É o 'chefe' da aplicação. Fica responsável por criar a janela do jogo quando o usuário abre o .exe, controlar botões de minimizar/maximizar, salvar arquivos no disco do computador e, principalmente, iniciar o servidor Java (backend) que roda em background.
- Departamento Visual (Renderer Process): É a interface que o usuário vê — toda a tela do jogo é desenhada aqui. É exatamente a mesma tecnologia usada em navegadores web (React). Não pode acessar o disco ou sistema operacional diretamente por questões de segurança.
- Departamento de Segurança (Preload Script): Atua como uma 'recepcionista' entre a interface visual e o chefe. Define quais operações do sistema o jogo pode fazer (ex: minimizar janela, verificar se o servidor começou) sem permitir acesso irrestrito ao computador.

7.2 O Que Acontece Quando o Jogo Abre (Fluxo de Inicialização)

Quando o usuário clica no arquivo .exe do Lorekeeper, vários passos ocorrem automaticamente em poucos segundos. O processo foi otimizado para ser rápido e inteligente:

- 1. O computador executa o programa .exe. O 'chefe' (Main Process) acorda e começa a trabalhar.
- 2. O programa verifica: é a primeira vez que alguém está jogando? Se sim, vai demorar um pouco para preparar tudo (criar banco de dados). Se não, abre rápido.
- 3. Antes de começar o servidor do jogo, o programa verifica se a 'porta 8000' (um caminho que o servidor usa para receber requisições) não está ocupada. Se estiver, limpa.
- 4. O servidor Java (backend) é iniciado em background, com configurações para usar pouca memória (ideal para computadores mais simples).
- 5. O programa aguarda o servidor ficar pronto, checando a cada meio segundo (máximo 60 segundos).
- 6. Uma vez que o servidor está pronto, a janela visual do jogo abre e a interface React é exibida (mesma do navegador).

- 7. O jogo envia um sinal dizendo 'estou pronto para receber dados' e aí tudo começa a funcionar.

7.3 Otimizações para Rodar Rápido

Para que o jogo rode bem mesmo em computadores mais antigos ou com menos recursos, foram aplicadas várias otimizações:

- Renderização clara em alta definição: O programa detecta a resolução da tela e ajusta a qualidade para não ficar borrado.
- Uso eficiente de processamento: A GPU (processador gráfico) é usada para desenhar a interface, deixando o processador principal livre.
- Uso controlado de memória: O servidor Java recebe limites (usar no máximo 256 MB), evitando que o jogo consuma toda a RAM do computador.
- Boot rápido na segunda vez: Na primeira abertura, pode demorar 5-10 segundos. Nas aberturas seguintes (quando o banco já existe), abre em ~500 milissegundos.

7.4 O Servidor Incorporado (Backend Java)

Uma das vantagens principais da versão desktop é que o servidor do jogo (a 'máquina' que processa combates, calcula XP, gerencia saves) não roda em um servidor remoto na internet. Ele vem embutido no arquivo .exe, rodando no próprio computador do jogador.

- O que é: Um arquivo chamado 'backend.jar' que contém todo o servidor Java do jogo.
- Onde fica: Vem dentro do arquivo .exe. Quando o jogo abre, esse arquivo é descompactado e executado em background.
- Banco de dados: Os dados do jogo (personagens, progresso, saves) são salvos em um arquivo SQLite (banco de dados leve) no computador do jogador. Não precisa de PostgreSQL instalado.
- Vantagem: O jogador pode jogar sem internet (após abrir o jogo uma vez). Tudo roda localmente.
- Mesma lógica: O servidor é idêntico ao da versão web, garantindo que o jogo se comporte igual.

7.5 Como a Interface Conversa com o Sistema (Comunicação IPC)

A interface visual (React) precisa solicitar ao 'chefe' (Main Process) para fazer operações no computador, como minimizar a janela ou saber se o servidor já iniciou. Para isso, usamos um sistema seguro de mensagens chamado IPC (Inter-Process Communication).

- Minimizar, Maximizar, Fechar Janela: Quando o jogador clica nos botões de controle da janela, a interface manda mensagem para o chefe fazer essas operações.

- Verificar o estado da janela: A interface pode perguntar 'a janela está no modo tela cheia?' e receber resposta instantaneamente.
- Avisar que o Servidor Começou: O chefe fica checando se o servidor Java está pronto. Quando está, manda aviso para a interface, que pede ao jogador para começar.
- Confirmação de Recebimento: A interface responde ao chefe dizendo 'recebi a mensagem, tudo certo'.

7.6 Como os Dados são Guardados (Armazenamento Local)

Todos os dados do jogo ficam salvos no computador do jogador, em um arquivo de banco de dados chamado SQLite. Diferente da versão web, que depende de um servidor remoto, a versão desktop armazena tudo localmente.

- Salvos do Jogo: Todos os 3 slots de save ficam no disco do computador. Se o jogador desligar o PC ou fechar o jogo, o progresso é restaurado quando volta a jogar.
- Configurações: Preferências do jogador (volume, brilho, linguagem) também ficam salvas.
- Token de Autenticação: Para o jogador não precisar fazer login toda vez, um código especial é guardado que permite entrar automaticamente.
- Privacidade: Como tudo fica no computador do jogador, nenhum dado é enviado a um servidor na internet (exceto o comunicação local com o servidor Java embutido).

7.7 Criando o Arquivo executável (Build e Empacotamento)

Quando os desenvolvedores querem criar um arquivo .exe do jogo para distribuir, eles usam um processo automatizado que compila e embrulha tudo junto:

- Nome do Jogo: 'Lorekeeper contra as sombras da ignorância' aparece no executável e no instalador (se houver).
- Ícone: Uma imagem bonita que representa o jogo é associada ao .exe, aparecendo na área de trabalho e no menu Iniciar.
- Conteúdo Incluído: A interface visual (React), o servidor Java (backend.jar), e tudo que precisa é embalado junto no arquivo .exe.
- Tipo de Entrega: Pode ser um .exe simples (basta executar) ou um instalador. O Lorekeeper usa a versão simples (portável).
- Processo Automatizado: Com um comando no terminal, o computador automaticamente compila todo o código, testa, otimiza e cria o .exe final em uma pasta chamada 'release'.

7.8 Como os Desenvolvedores Testam Durante o Desenvolvimento

Enquanto o código está sendo desenvolvido, os programadores precisam testar rapidamente e ver mudanças em tempo real. Para isso, eles executam dois servidores simultaneamente:

- Servidor Visual (Vite): Roda a interface React e atualiza automaticamente conforme o programador digita o código (sem precisar fechar e reabrir).
- Electron + Backend: O jogo é aberto como um .exe de desenvolvimento e o servidor Java é iniciado normalmente, permitindo testar tudo junto.
- Resultado: Os desenvolvedores veem mudanças quase instantaneamente, acelerando o processo de desenvolvimento.

7.9 Entregando o Jogo Final (Distribuição)

Quando tudo está pronto, o arquivo final fica em uma pasta chamada 'release' contendo:

- Um arquivo .exe único: O usuário final simplesmente baixa esse arquivo e clica para executar. Sem necessidade de instalação ou configurações extras.
- Incluso: O .exe já contém o jogo, a interface visual e o servidor Java embutido. Pronto para jogar.

7.10 Protegendo o Computador do Jogador (Segurança)

Como o .exe roda no computador do jogador e tem acesso a arquivos, foram implementadas várias proteções de segurança para evitar problemas:

- Isolamento: A interface visual não pode acessar diretamente o disco ou sistema. Precisa pedir permissão para o 'chefe'.
- Controle de Acesso: O jogo só pode fazer operações específicas (minimizar, maximizar) e nada mais. Não consegue deletar arquivos ou instalar programas.
- Validação: Todos os comandos enviados entre a interface e o sistema são verificados para garantir que é legítimo.
- Sem risco: O código JavaScript que roda na interface (React) não consegue executar comandos perigosos diretamente.

7.11 Encerrando o Jogo Corretamente (Shutdown)

Quando o jogador fecha o .exe, o programa precisa encerrar de forma segura para não deixar nada rodando em background:

- Aviso: Antes de fechar completamente, o programa verifica se o servidor Java ainda está rodando.
- Desligamento Limpo: Se o servidor estiver ativo, o programa manda um sinal pedindo para ele encerrar graciosamente.
- Esperando: Dá tempo para o servidor salvar dados e fechar conexões.

- Força Máxima: Se o servidor não responder em tempo, o programa força o encerramento.
- Limpeza Final: Todos os arquivos temporários e processos são removidos, deixando o computador limpo.

8. Como o Código está Organizado (Implementação)

O Lorekeeper é dividido em duas grandes partes: a interface visual que o jogador vê (Frontend) e o 'motor' que processa tudo nos bastidores (Backend). Ambas as partes têm código bem organizado em pastas e arquivos específicos, facilitando a manutenção e novos desenvolvimentos.

8.1 A Interface Visual (Frontend - React e TypeScript)

O código que desenha a tela do jogo está organizado por funcionalidade. Pense como em um prédio com andares especializados:

- Ala 'app': O ponto de entrada — onde tudo começa e o estado global é gerenciado.
- Ala 'assets': Todas as imagens do jogo (personagens, inimigos, backgrounds, ícones) bem organizadas por tipo.
- Ala core/ — O cérebro do sistema: contém os contextos globais que todo o jogo consulta (autenticação do jogador, estado da partida, controle de tela cheia e tutorial interativo). Pense como a central de comando que mantém as decisões importantes acessíveis a qualquer tela.
- Ala features/ — Os andares do jogo: cada funcionalidade é um módulo independente com sua própria tela, componentes e lógica interna. O sistema de batalha não se mistura com o quiz; o menu de pausa não depende da loja. As funcionalidades incluem: autenticação (auth), batalha (battle), hub central (hub), seleção de personagem (character), menu principal e pausa (menu), missões (quests), quiz (quiz) e sistema de salvamento (save-system).
- Ala shared/ — As ferramentas comuns: código compartilhado por duas ou mais funcionalidades: componentes visuais genéricos (Layout, Barra de Título, Alerta Medieval), serviços de comunicação com o servidor, tipos de dados, constantes de imagens e utilitários. É como o almoxarifado do prédio — tudo que qualquer andar pode precisar está aqui.

Para manter a organização limpa, o projeto usa atalhos de importação (@app, @core, @features, @shared, @assets) — assim um componente em qualquer profundidade importa outro sem caminhos longos como ../../../../components/Botão.

8.2 O Motor do Jogo (Backend - Spring Boot)

O Backend (servidor Java) é o 'cérebro' do jogo — processa batalhas, calcula XP, valida respostas de quiz e guarda dados. Está organizado também em partes especializadas:

- Porteiros (Controllers): Recebem pedidos da interface (ex: 'começar uma batalha?') e repassam para o pessoal certo. Depois enviam resposta em formato que a interface entende (JSON).
- Especialistas (Services): Fazem o trabalho real — calculam dano em combate, determinam se uma resposta está correta, gerenciam regras de quests.
- Arquivo (Repositories): Acessam o banco de dados para salvar e recuperar informações (personagens, saves, histórico de combates).
- Modelos de Dados: Definem que informações cada coisa tem (um personagem tem nome, classe, HP, XP; um combate tem dois personagens, turno atual, resultado).
- Controle de Segurança (Security): Valida se o jogador realmente é quem diz ser, usando tokens especiais (JWT) para não precisar fazer login a cada requisição.
- Configurações: Definem se o backend vai usar PostgreSQL (versão web) ou SQLite (versão desktop), como lidar com requisições de origens diferentes etc.

8.3 Um Exemplo Prático: Como Funciona uma Batalha

Para entender como frontend e backend trabalham juntos, vamos acompanhar o que acontece quando o jogador começa uma batalha:

- 1. Jogador clica 'Iniciar Batalha': A interface envia uma mensagem para o servidor pedindo para criar uma batalha com seu personagem contra um inimigo.
- 2. Servidor válida: Verifica se o personagem existe, se o inimigo é válido etc. Cria a batalha no banco de dados.
- 3. Servidor responde: Envia para a interface as informações da batalha (HP do personagem, HP do inimigo, primeira pergunta, etc).
- 4. Interface exhibe: A tela de batalha aparece com animações, HP bars e a primeira pergunta.
- 5. Jogador responde: Clica em uma das alternativas.
- 6. Interface envia resposta: Manda para o servidor qual alternativa foi escolhida.
- 7. Servidor calcula: Determina se a resposta está correta. Se sim, o ataque do jogador conecta. Se não, o inimigo contra-ataca.
- 8. Servidor atualiza: Recalcula HP de ambos, seleciona a próxima pergunta, verifica se alguém morreu.
- 9. Servidor responde: Envia todo o novo estado para a interface.
- 10. Interface atualiza: Mostra animação de dano, atualiza HP, exhibe a próxima pergunta.
- 11. Loop: Repete até que alguém vença.

8.4 Interface Gráfica e Resultados Visuais

Nesta seção, é apresentado o resultado visual e interativo da arquitetura de **Frontend** detalhada no tópico 8.1. As interfaces do **Lorekeeper** foram projetadas para unir a identidade visual imersiva de um RPG clássico de fantasia à clareza utilitária necessária para um software educacional. A seguir, são exibidas as telas fundamentais da jornada do jogador. Elas demonstram não apenas o fluxo de entrada, mas também a integração do combate com o painel de estatísticas da conta, que enfatiza o acompanhamento do desempenho no aprendizado do idioma.

8.4.1 Módulo de Autenticação e Perfil

O contato inicial do usuário com o sistema ocorre por meio do módulo de autenticação. As telas de Login e Cadastro foram desenhadas para oferecer um acesso direto, conectando-se à API REST para validação de credenciais e geração do **token** JWT. Para garantir a segurança sem quebrar a imersão, a interface utiliza painéis flutuantes sobrepostos a elementos temáticos do jogo.

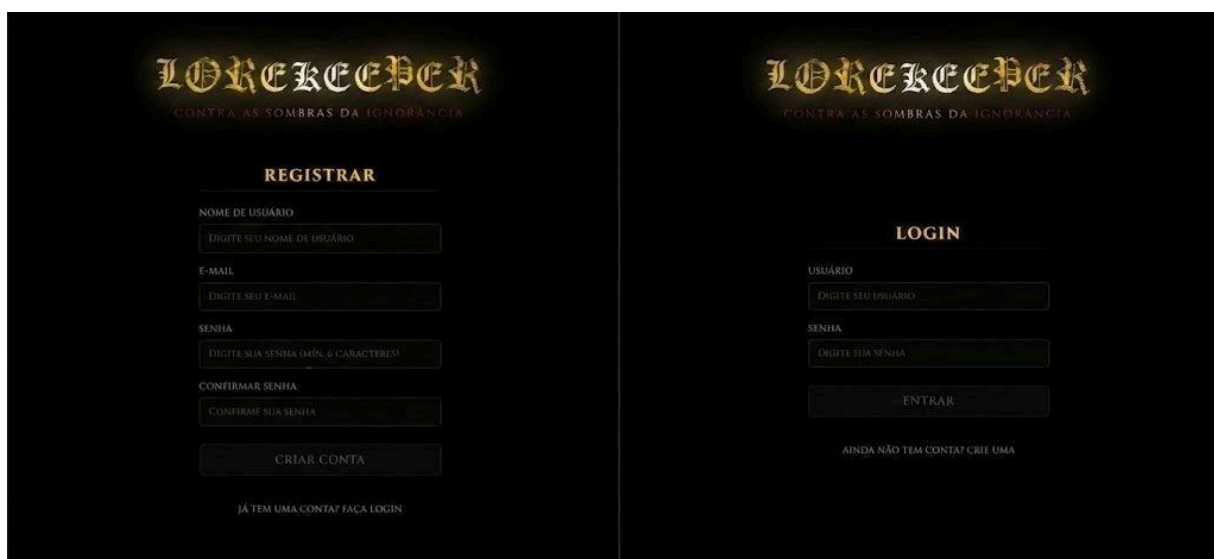


Figura 1 Interfaces de autenticação e registro de novos aventureiros.

Após o acesso bem-sucedido, o jogador é direcionado ao Menu Principal (Figura 2). Este atua como a central de controle do sistema ('Lobby'), permitindo ao usuário iniciar uma nova aventura (Seleção de Classe), continuar sua jornada de onde parou ou gerenciar sua conta e preferências. As telas secundárias acessadas por aqui, como as Configurações e a Criação de Personagem, podem ser consultadas em detalhe no Apêndice C.

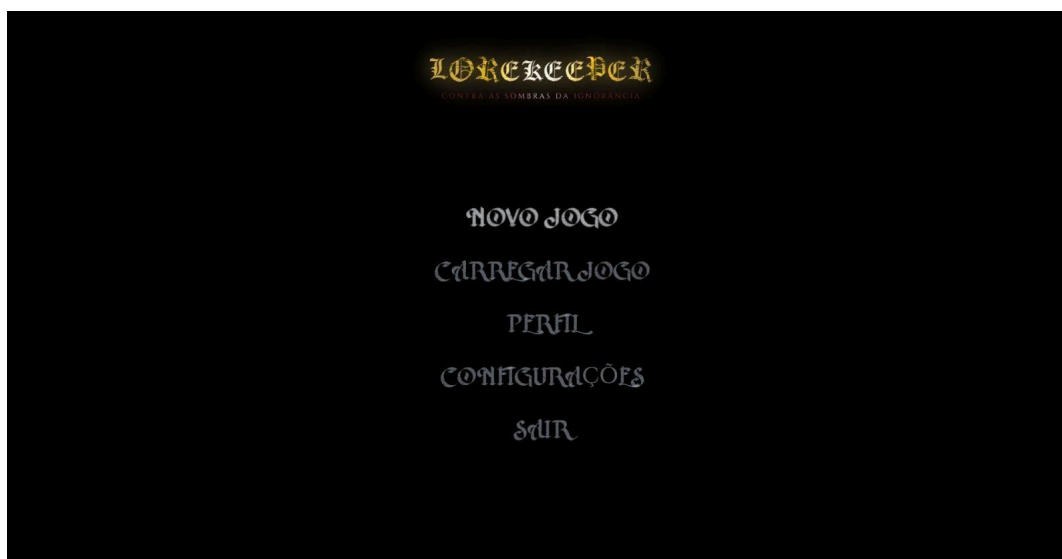


Figura 2 Menu Principal, a central de acesso a todas as funcionalidades do sistema e jogos salvos.

Acessando o perfil a partir do Menu Principal, o jogador encontra o seu 'Registro da Jornada' (Figura 3). Diferente de sistemas tradicionais de gestão de conta, este painel reflete o duplo propósito do Lorekeeper, consolidando em um mesmo espaço as proezas de combate (inimigos derrotados, ouro e experiência) e os indicadores de evolução cognitiva no aprendizado de inglês (duelos de idioma resolvidos, taxa de acurácia e vocabulário descoberto).

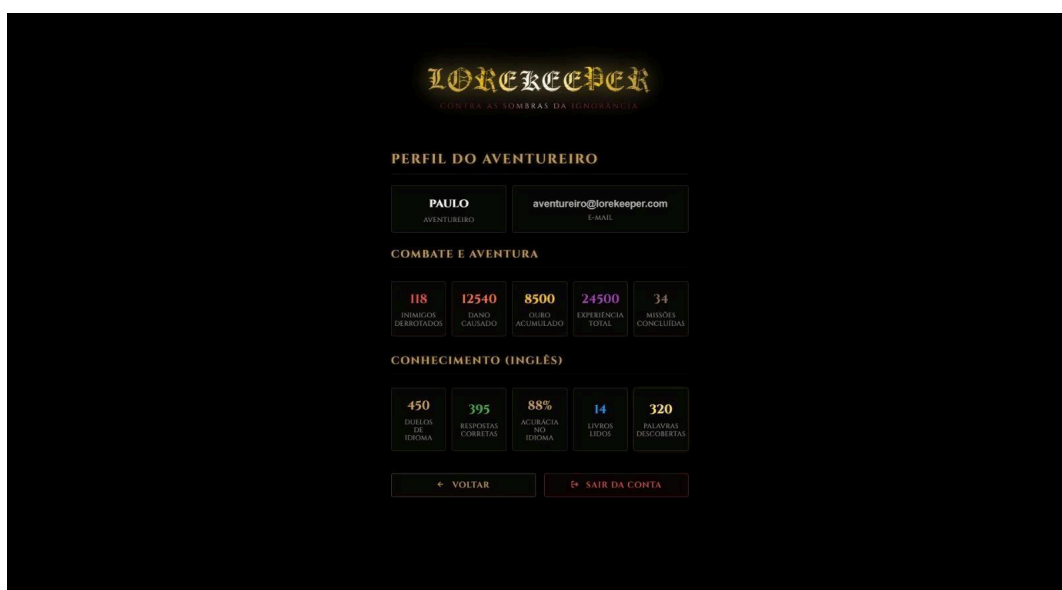


Figura 3 Painel do usuário destacando o equilíbrio entre métricas de aventura e de aprendizagem

8.4.2 Exploração e Combate Educativo

Ao iniciar um Novo Jogo ou Carregar um jogo salvo, o jogador é transportado para o

Hub Central (Figura 4). Este ambiente in-game atua como o ponto focal para todas as atividades mecânicas e pedagógicas da jornada, onde o jogador pode interagir com instalações como a Biblioteca (para estudo prévio) e a Arena (para iniciar embates).



Figura 4 Visão geral do Hub Central, espaço do mapa para preparação e escolha de atividades.

O núcleo da experiência interativa ocorre na interface de Batalha (Figura 5) Nela, a HUD (Heads-Up Display) foi otimizada para manter os elementos vitais do RPG (pontos de vida, opções de ataque) nas laterais, garantindo que o centro da tela permaneça limpo para a aparição do componente educacional.

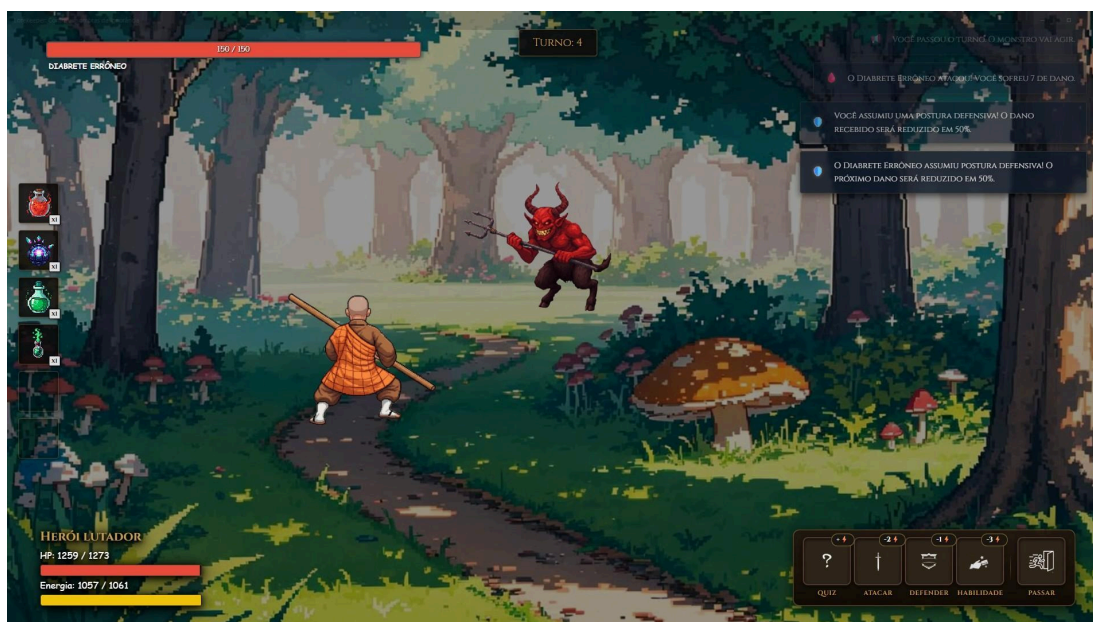


Figura 5 Interface de combate por turnos aguardando a decisão tática do jogador.

Por fim, ao selecionar uma ação tática, o sistema invoca o Duelo de Idioma. A tela foca a atenção do jogador no desafio pedagógico: a resolução correta da pergunta em inglês determina o sucesso e a eficácia da ação no RPG, validando o conceito de 'combate educativo' estabelecido nos requisitos do sistema.



Figura 6 Modal de desafio linguístico (**Quiz**) integrado ao fluxo de combate.

9. Como Garantir Que o Jogo Funciona (Testes e Qualidade)

Antes de entregar o jogo aos jogadores, garantimos que tudo está funcionando corretamente. Para isso, usamos vários tipos de testes em diferentes níveis.

9.1 Testando o Motor (Backend)

- Testes de Lógica: Verificam se os cálculos estão corretos (se alguém responde certo, perde HP? Se sobe de nível, ganhou atributos novos?). Cada regra do jogo é testada isoladamente.
- Testes de Integração: Verificam se as partes trabalham bem juntas (se salvo o jogo, depois consigo carregar?). Testam endpoints (portas de comunicação) que a interface usa.
- Testes de Segurança: Verificam se um jogador consegue ver dados de outro jogador ou trapacear.
- Cobertura: Meta de 70% do código testado automaticamente.

9.2 Testando a Interface (Frontend)

- Verificação de Estilo (ESLint): um programa automático lê o código e avisa se há erros ou más práticas (variáveis sem usar, dependências faltando, padrões do React não seguidos).
- Verificação de Tipos (TypeScript): o TypeScript, no modo mais rigoroso, avisa imediatamente se tentarmos usar um número como texto ou acessar uma propriedade que não existe.
- Testes de Componentes (Vitest + React Testing Library): planejado para a próxima etapa. Verifica se cada peça visual funciona sozinha — botões reagem ao clique, formulários aceitam entrada, barras de vida atualizam corretamente. Roda em Node.js, sem precisar abrir o navegador.
- Testes de Cenários Completos (Playwright): planejado para a próxima etapa. Um navegador de verdade, controlado por código, simula um jogador real percorrendo o jogo inteiro — login, criar personagem, batalha, quiz, salvar — e confere se cada etapa produz o resultado esperado na tela.
- Teste em Diferentes Telas (Playwright): planejado para a próxima etapa. O jogo é aberto em três tamanhos de tela (desktop 1920x1080, notebook 1366x768 e tablet 768x1024) para verificar se a interface se adapta corretamente em cada resolução.

9.3 Testando Integração (Frontend + Backend)

- Simulação: Durante o desenvolvimento, usamos dados fake (simulados) para não depender do servidor. Em testes mais sérios, usamos um servidor real.
- Contrato: Ambos frontend e backend combinam que tipo de mensagem vão trocar. Se isso mudar, os testes avisam.
- Timeout: Verifica se a interface não congela se o servidor demorar para responder.

9.4 Testando a Versão Desktop (Electron)

- Abertura: Verifica se o arquivo executável abre, a janela aparece e o jogo começa.
- Comunicação: Testa se os botões de minimizar/maximizar funcionam, se o programa recebe avisos do servidor corretamente.
- Servidor Embarcado: Verifica se o JAR inicia rápido (menos de 5 segundos na segunda vez), se o banco de dados local é criado e acessado.
- Saves no Disco: Testa se conseguir salvar e depois carregar um jogo sem perder dados.
- Segunda Abertura: Abre o jogo, fecha, abre novamente — deve ser rápido e reconhecer o banco existente.

9.5 Testes Manuais

Cenários exploratórios críticos (web e desktop):

- Login: Credenciais corretas → sucesso; incorretas → erro claro.

- Criação de personagem: Seleção de classe, atributos iniciais corretos.
- Batalha: Pergunta exibida, resposta correta → sucesso, incorreta → falha.
- Recompensas: XP, ouro, itens distribuídos conforme regras.
- Quests: Aceitar, progredir, completar, receber recompensas.
- Achievements: Desbloquear automaticamente por eventos.
- Save/Load: 3 slots funcionam, estado restaurado completo.
- Tutorial: Apenas primeira vez (controlável).
- (Desktop) Electron: .exe inicia, tudo local, sem dependência de servidor remoto.

10. Conclusão

Lorekeeper materializa uma arquitetura web moderna e extensível para RPG educativo focado em idiomas. A documentação técnica consolida requisitos, desenho em camadas e mecânicas para orientar evolução, padronização do código e distribuição desktop com Electron.

Referências

- COSTA, B. da S. da. O poder dos games: a influência na ampliação do vocabulário em língua estrangeira / The power of games: the influence on expanding vocabulary in the foreign language. 2024. 42 f. Trabalho de Conclusão de Curso (Licenciatura Plena em Letras — Língua Portuguesa, Língua Inglesa e respectivas literaturas) — Universidade Estadual do Maranhão, Campus Santa Inês, Santa Inês, MA, 2024. Orientação de Robson de Macêdo Cunha. Disponível em: <https://repositorio.uema.br/jspui/handle/123456789/2467>. Acesso em: 8 abr. 2026.
- SILVA, F. W. da C.; TOASSI, P. F. P. O papel dos jogos eletrônicos na aquisição da língua inglesa / The role of electronic games in the learning of English. Revista do GEL, Fortaleza, v. 17, n. 1, p. 259–283, 2020. DOI: <https://doi.org/10.21165/gel.v17i1.2757>.

Apêndice A — Nota sobre arquitetura alvo vs. evolução do repositório

Este documento descreve a arquitetura recomendada e já parcialmente refletida no código (Seção 8). A integração com Electron para distribuição desktop (Seção 7) é plenamente operacional e permite entrega .exe standalone para Windows. Refatorações futuras (padronização de pastas feature-based no front, camada anti-corrupção para DTOs, documentação OpenAPI gerada) devem manter os mesmos limites entre cliente, API e persistência aqui definidos. A estratégia de testes (Seção 9) inclui expansão de cobertura automatizada e validação de performance em produção.

Apêndice B — Dicionário de Termos Técnicos

Este apêndice define termos técnicos utilizados neste documento, facilitando a compreensão para leitores não técnicos:

API (Interface de Programação de Aplicações): Um 'contrato' que define como dois programas conversam entre si. A interface visual faz requisições à API do servidor, que responde com os dados solicitados.

Backend: O 'motor' do jogo que roda no servidor. Processa batalhas, calcula XP, valida respostas de quiz e gerencia o banco de dados.

JWT (Token de Segurança): Um código especial que o servidor dá ao jogador quando ele faz login. O jogador envia este código em cada requisição para provar que está autenticado, sem precisar enviar senha toda vez.

React: Biblioteca JavaScript que facilita criar interfaces visuais interativas. É o que desenha a tela do jogo que o jogador vê.

Spring Boot: Framework que facilita criar um servidor (backend) em Java rapidamente, com tudo que precisa para funcionar (segurança, banco de dados, HTTP).

TypeScript: Versão melhorada de JavaScript que avisa erros antes do código rodar. Ajuda a evitar bugs.

PostgreSQL: Banco de dados (como um arquivo organizado) que armazena informações dos jogadores, personagens, saves, combates, etc.

SQLite: Versão leve de banco de dados usada na versão desktop. Fica como arquivo no computador do jogador, sem precisar de um servidor.

Docker: Tecnologia que embrulha o servidor e banco de dados em um 'computador virtual', garantindo que funcionem igual em qualquer máquina.

Git: Sistema de controle de versão. Guarda histórico de todas as mudanças no código, quem fez, quando e por quê.

Electron: Framework que permite embrulhar uma aplicação web em um executável (.exe) para rodar como um programa desktop tradicional.

Frontend: A interface visual do jogo que o jogador vê e interage. Roda no navegador web ou na janela do Electron.

HTTP/HTTPS: Protocolo que a interface usa para 'falar' com o servidor. HTTPS é a versão segura, que criptografa as mensagens.

JSON: Formato padronizado para enviar dados entre a interface e o servidor. Simples e fácil de ler.

Responsive Design: Técnica de fazer a interface se adaptar a diferentes tamanhos de tela (smartphone, tablet, desktop) sem perder funcionalidade.

Vite: Ferramenta que compila o código React em arquivos otimizados rápidos para o navegador.

ORM (Mapeamento Objeto-Relacional): Técnica que permite ao programador trabalhar com objetos (personagem, batalha) em vez de escrever SQL direto. Hibernate é um exemplo.

REST API: Estilo de design para criar APIs usando operações HTTP simples (GET para pedir, POST para enviar, PUT para atualizar, DELETE para remover).

Endpoint: Um 'ponto de acesso' específico da API. Ex: /api/battle/start é um endpoint para iniciar uma batalha.

Containerização: Prática de empacotar uma aplicação com todas suas dependências em um container (como Docker) para rodar em qualquer lugar.

Apêndice C (Telas Adicionais)

Este apêndice complementa a Seção 8.4, apresentando o conjunto completo de telas e modais do **Lorekeeper** que não foram incluídos no corpo principal do documento. As interfaces estão organizadas por contexto de funcionalidade e acompanhadas de uma breve descrição de sua finalidade:

C.1 — Novo Jogo e Carregamento

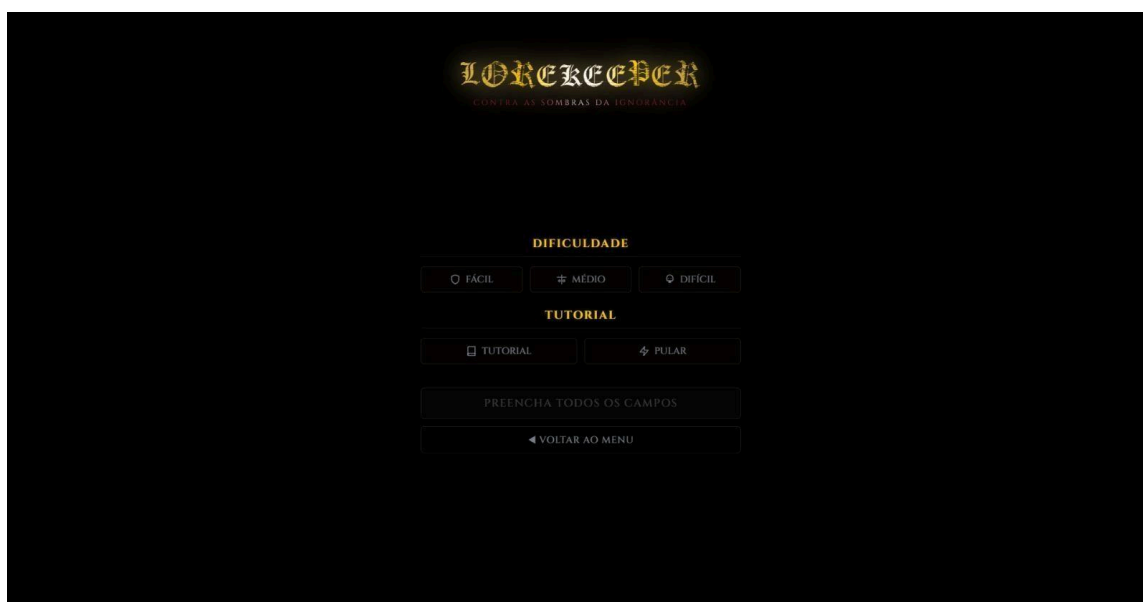


Figura C.1 Configuração de novo jogo: seletor de dificuldade (Fácil / Médio / Difícil), toggle para ativar/desativar tutorial, botão "INICIAR AVENTURA".

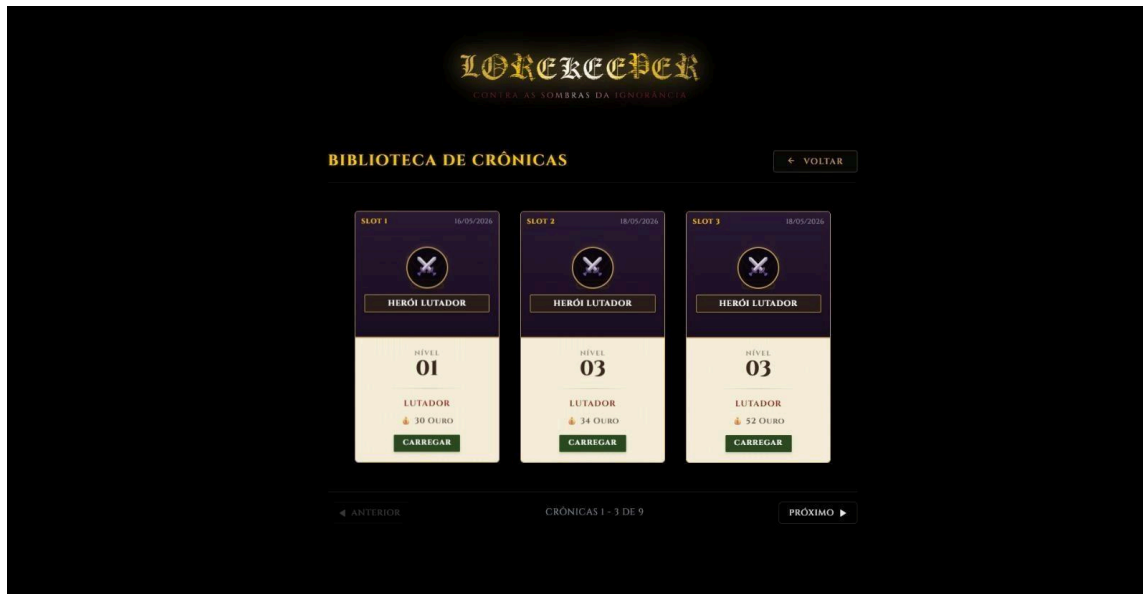


Figura C.2 grade paginada de slots de save (3 por página). Cada slot exibe: nome do personagem, classe, nível, ouro e data do save. Clique para carregar. Suporta modo overlay (via menu de pausa).

C.2 — Configurações, Pausa e Status

Figura C.3 a C.6

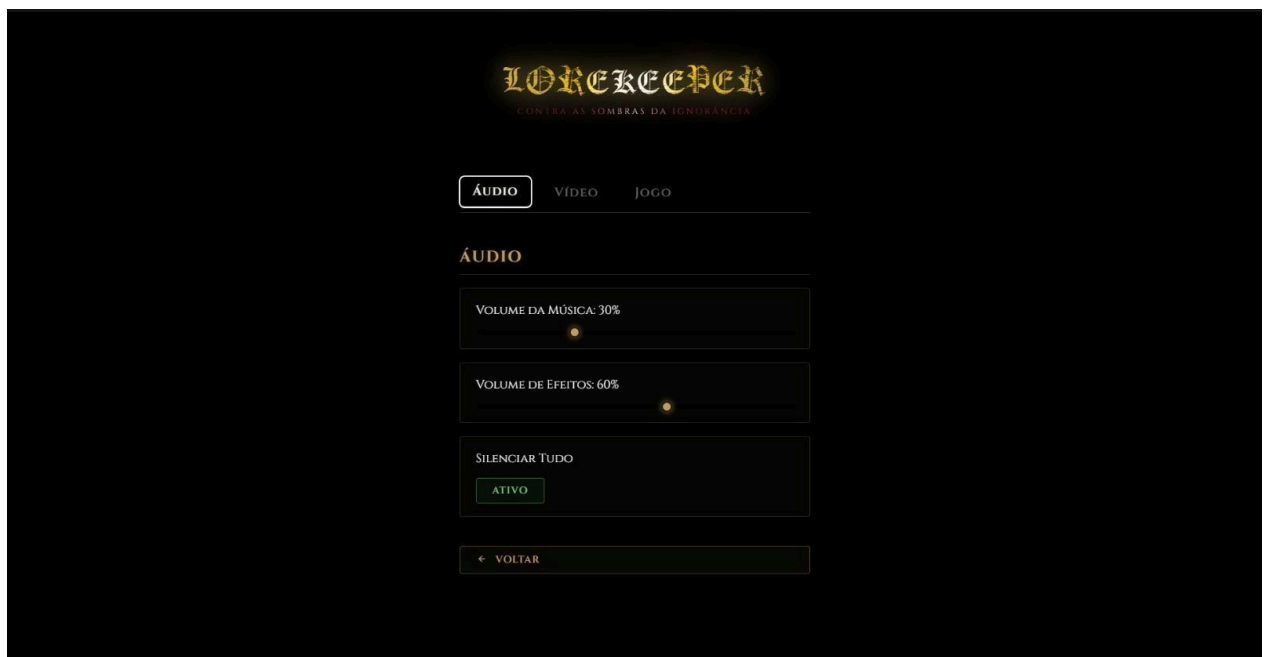


Figura C.3 Três abas: Áudio, Vídeo e Jogo.



Figura C.4 Overlay de pausa durante o jogo. Opções: "Continuar", "Status do Personagem", "Recuar para o Hub" (durante batalha), "Salvar Jogo", "Carregar Jogo", "Opções", "Sair para o Menu". Inclui QuestTracker embutido. Fecha com botão direito ou ESC.

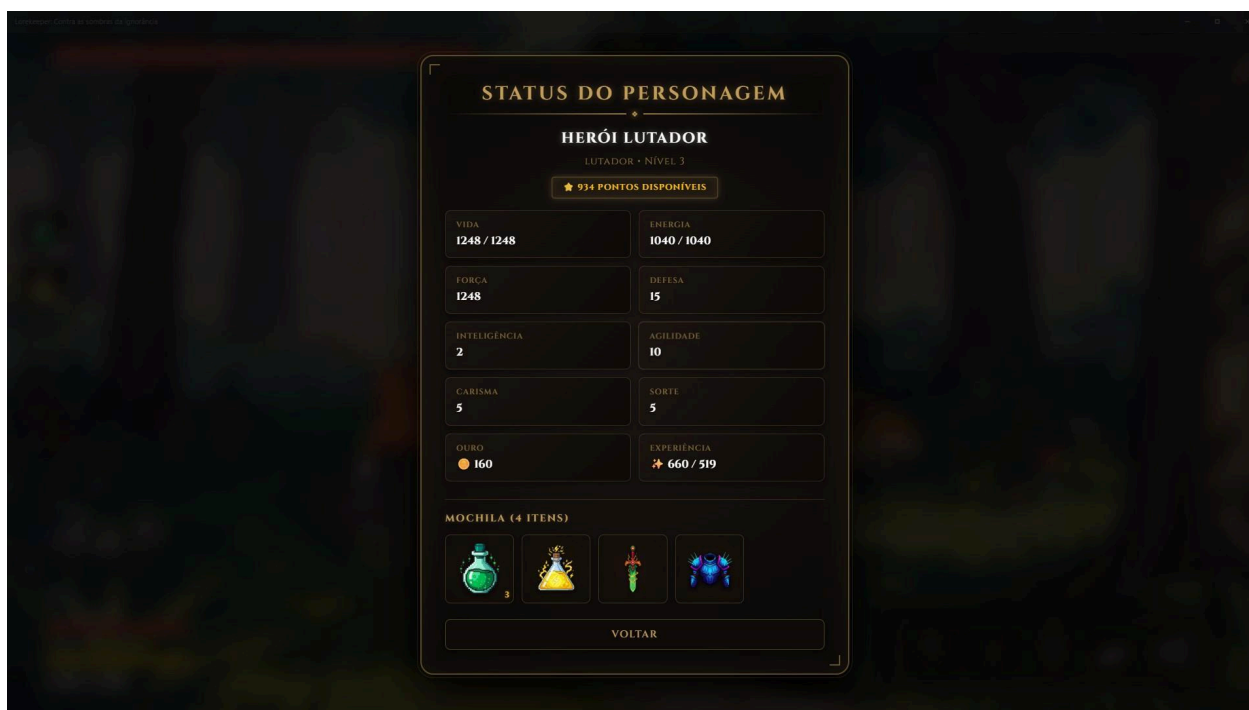


Figura C.5 Estatísticas detalhadas: Vida, Energia, Força, Defesa, Inteligência, Agilidade, Carisma, Sorte. Dados de progresso: Ouro, XP (com barra de progresso). Grade de inventário com ícones de itens equipados.

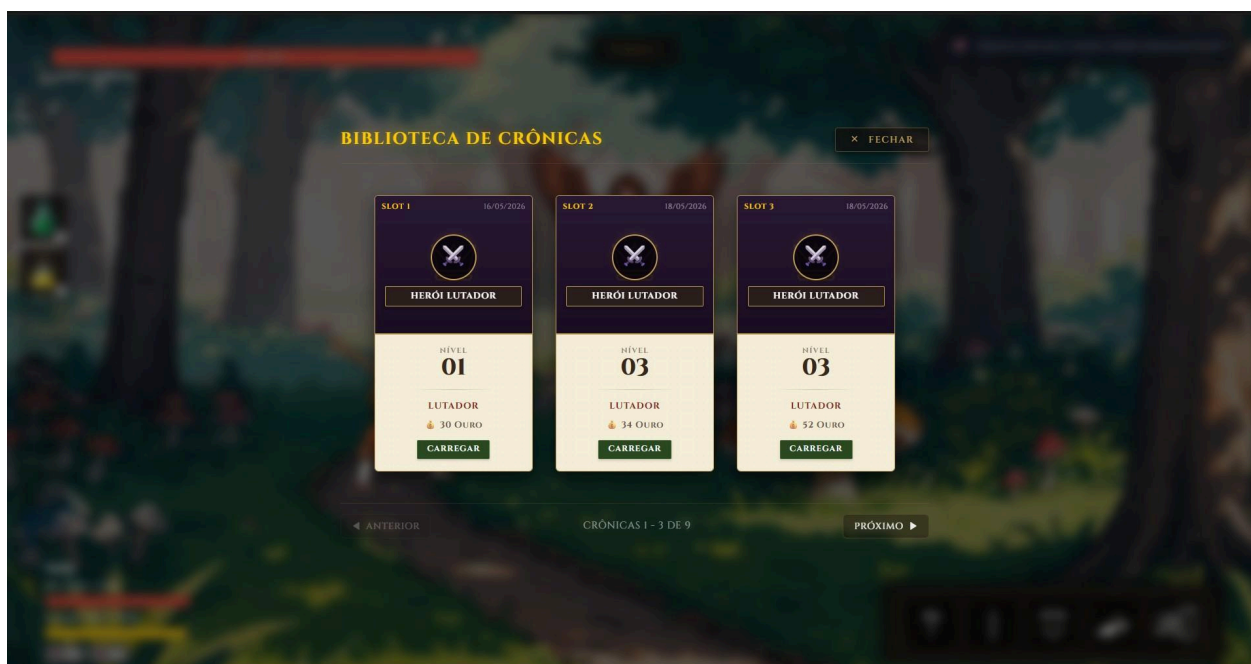


Figura C.6 Cartão individual de slot usado em Salvar e Carregar. Exibe: nome do slot, ícone da classe, nome do personagem, nível, barras de HP/Energia, ouro e data do save.

C.3 — Criação e Seleção de Personagem

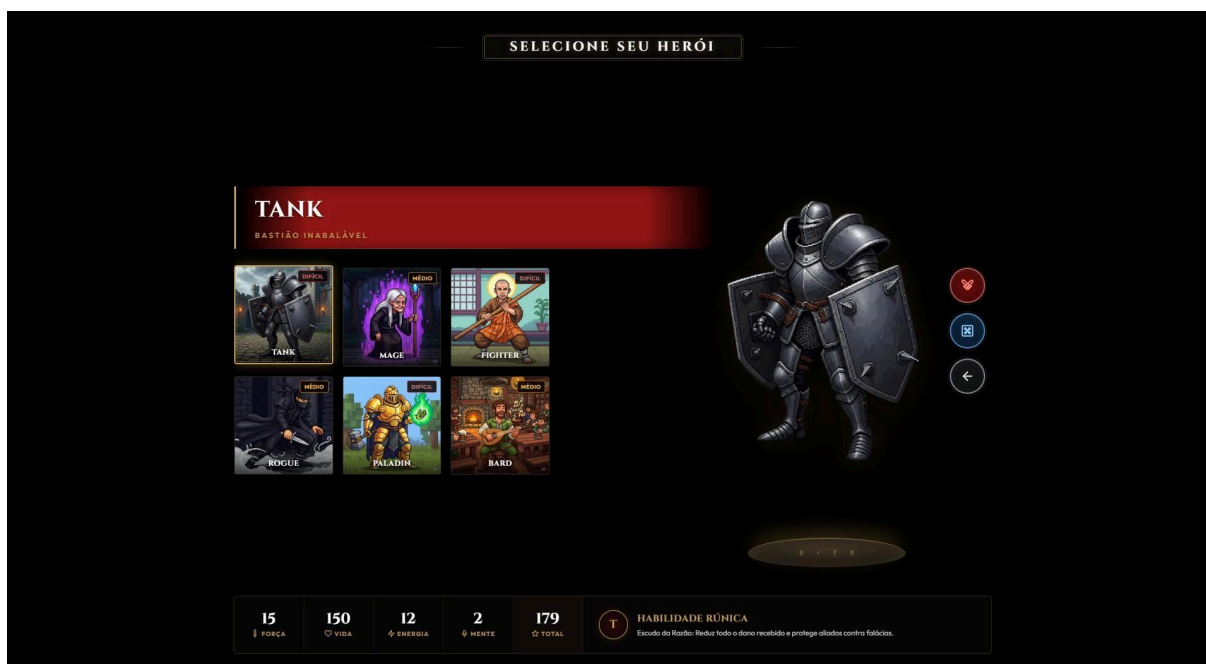


Figura C.7 Tela de criação de herói. Lado esquerdo: grade de cartas de classe com nomes e imagens. Lado direito: avatar flutuante do personagem sobre pedestal rúnico, com botões circulares. Parte inferior: colunas de atributos e brasão da habilidade especial.

C.4 — Hub: Torre do Conhecimento (Pisos F1–F4)

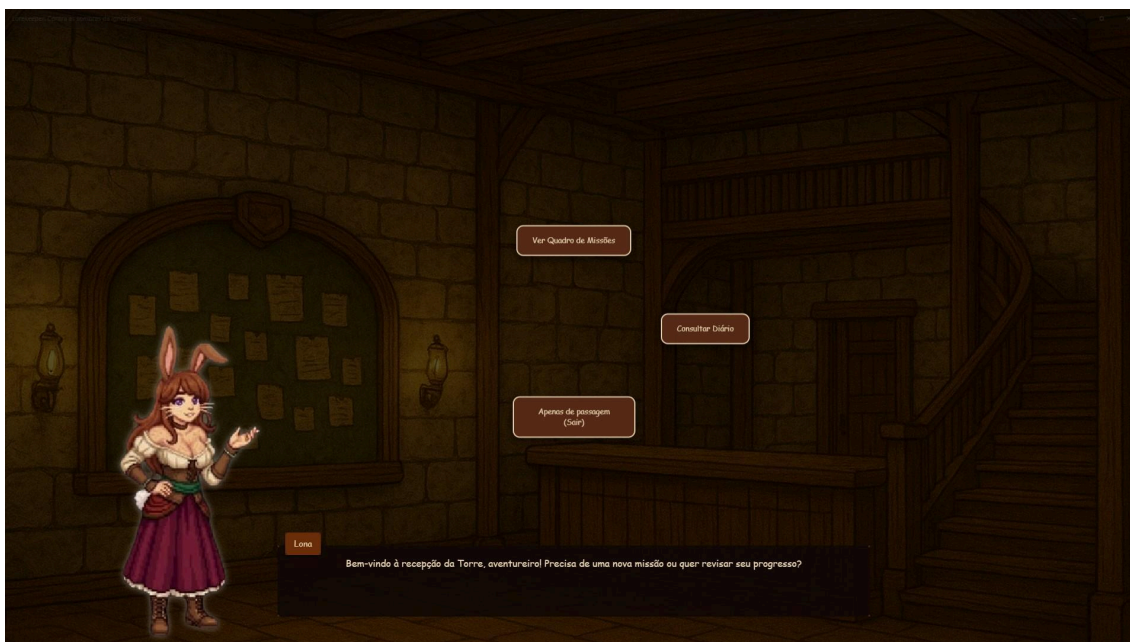


Figura C.8 F1 - Recepção (NPC Lona) como a recepcionista da guilda. Opções de diálogo: "Ver Quadro de Missões" ou "Consultar Diário".

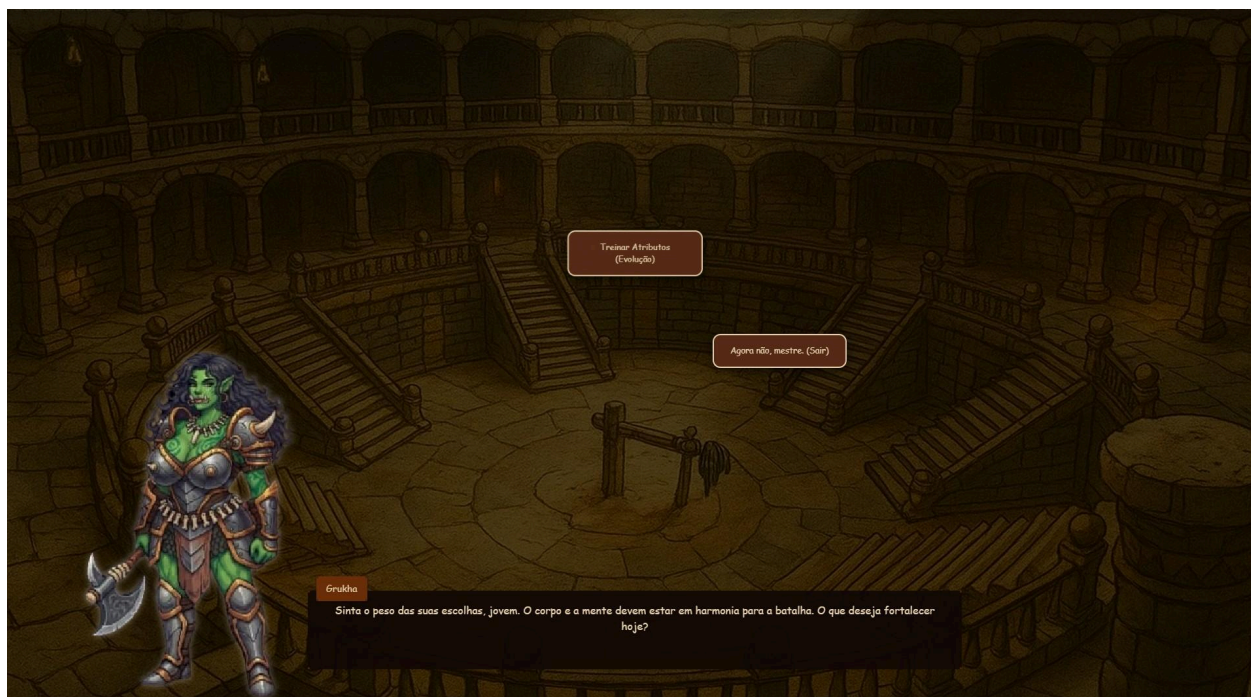


Figura C.9 F2 — Treinamento (NPC Grukha) como a mestre de treinamento. Opção: "Treinar Atributos (Evolução)" — abre o modal de evolução de status.

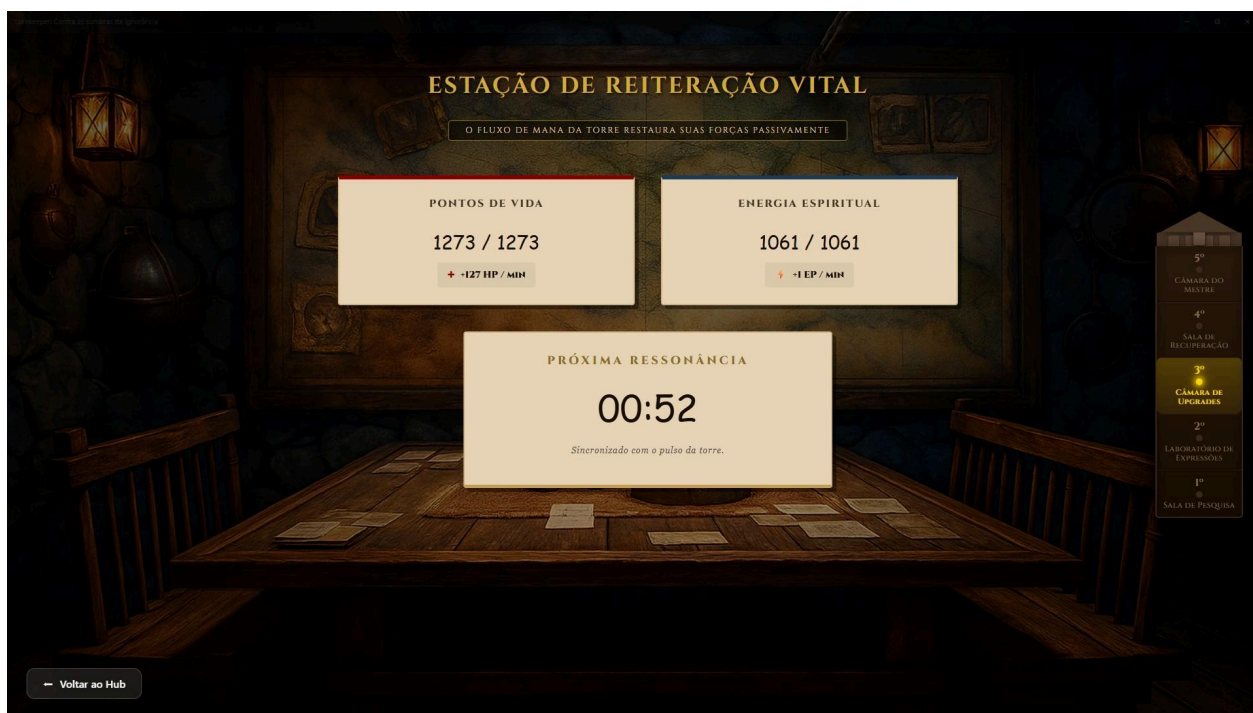


Figura C.10 F3 — Sala de Cura, Entrada automática na estação de cura. Overlay de carregamento enquanto os dados de HP/Energia são consultados no servidor.

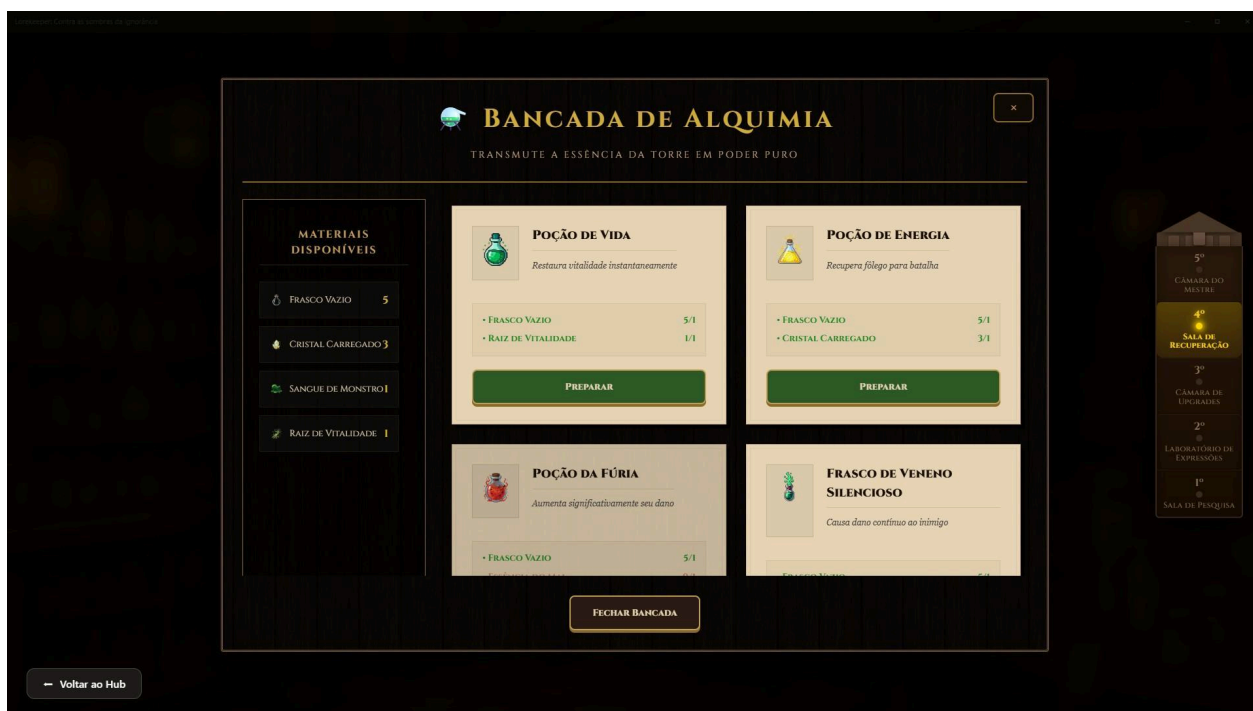


Figura C.11 F4 — Laboratório Mesa de crafting de alquimia. Ao clicar no ponto de interação, abre o modal overlay com: prateleira de ingredientes disponíveis, cartas de receita e funcionalidade de criação de poções a partir dos ingredientes.

C.5 — Hub: Arena, Biblioteca, Loja e Estalagem

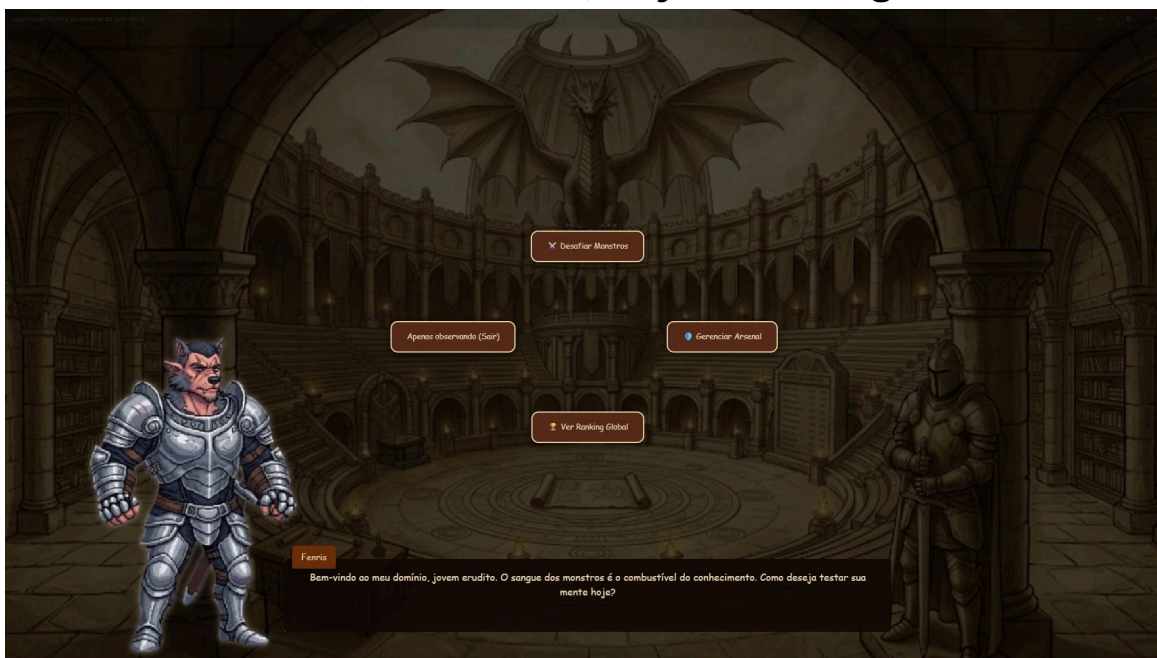


Figura C.12 Arena, NPC gladiador Fenris. Ao interagir, abre cutscene com opções: "Desafiar Monstros". "Gerenciar Arsenal", "Ver Ranking Global".

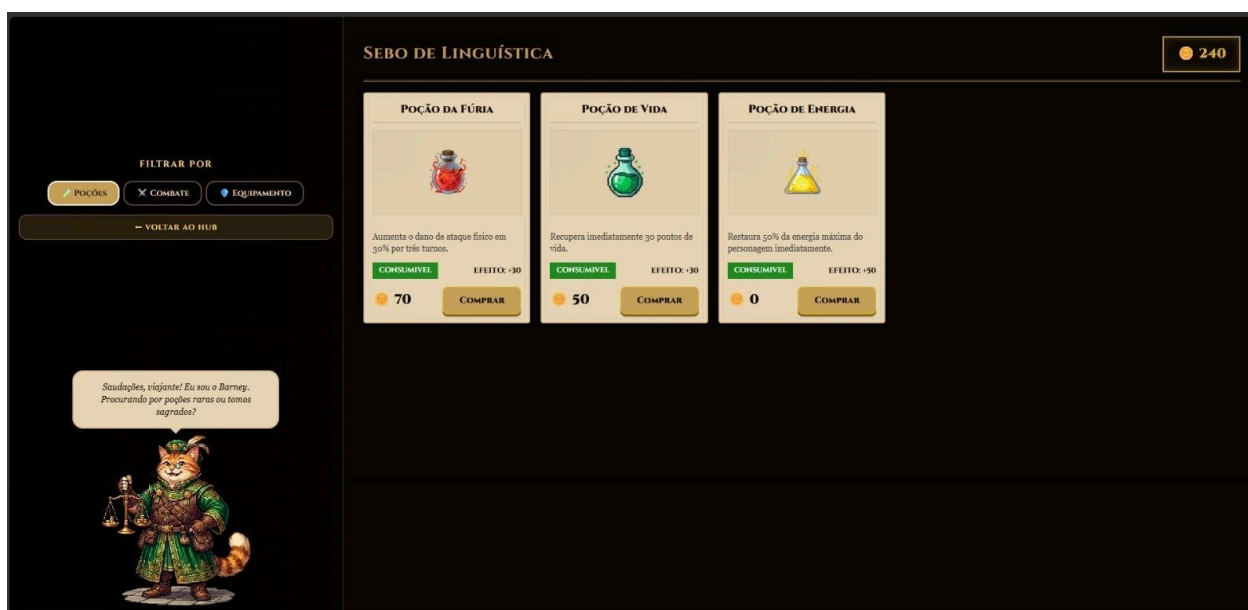


Figura C.13 Sebo de Linguística, Loja com abas de categoria (TUDO, POÇÕES, COMBATE, EQUIPAMENTO) e grade de cartas de itens. Cada carta exibe: nome, imagem, descrição, selo de tipo, efeito e preço. Botão "Comprar" com verificação de saldo de moedas. Exibe saldo atual do jogador.

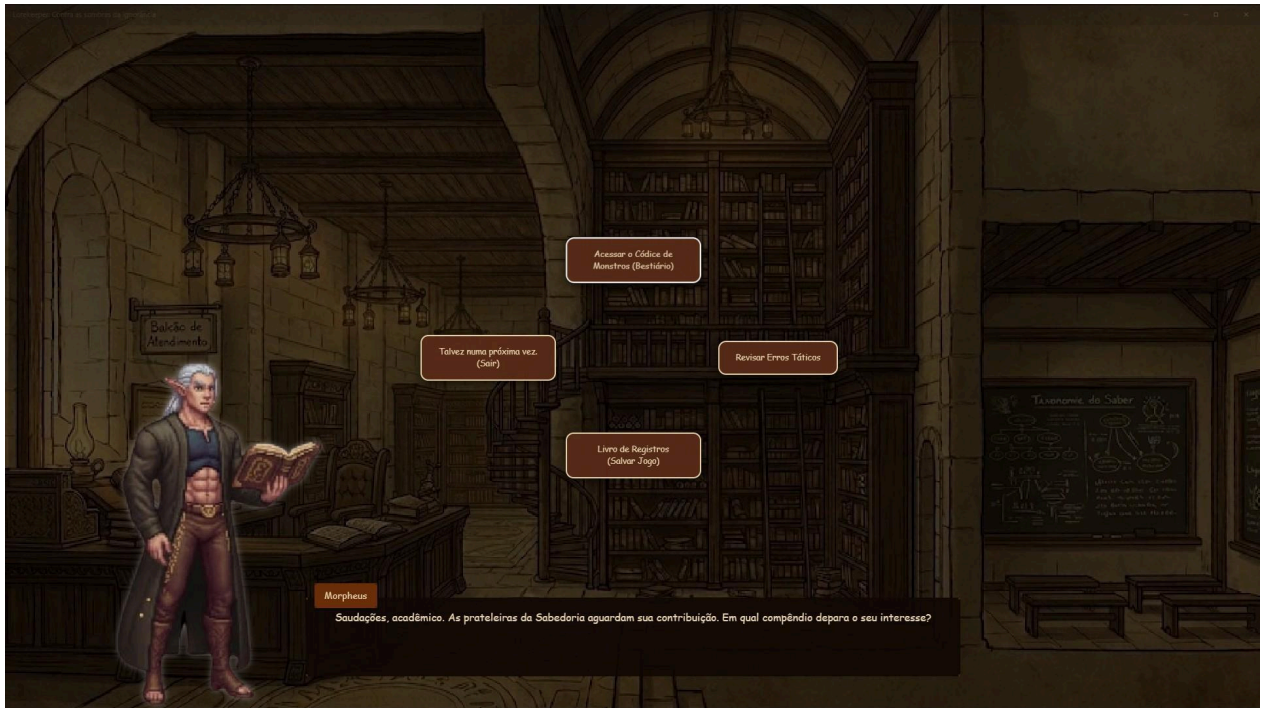


Figura C.14 Biblioteca, NPC bibliotecário Morpheus. Ao interagir, abre cutscene com opções: "Acessar o Códice de Monstros (Bestiário)", "Revisar Erros Táticos" (abre Sala de Revisão), "Livro de Registros (Salvar Jogo)".



Figura C.15 Mapa, outras áreas do mapa disponíveis futuramente, utilizadas para encontrar novos monstros.

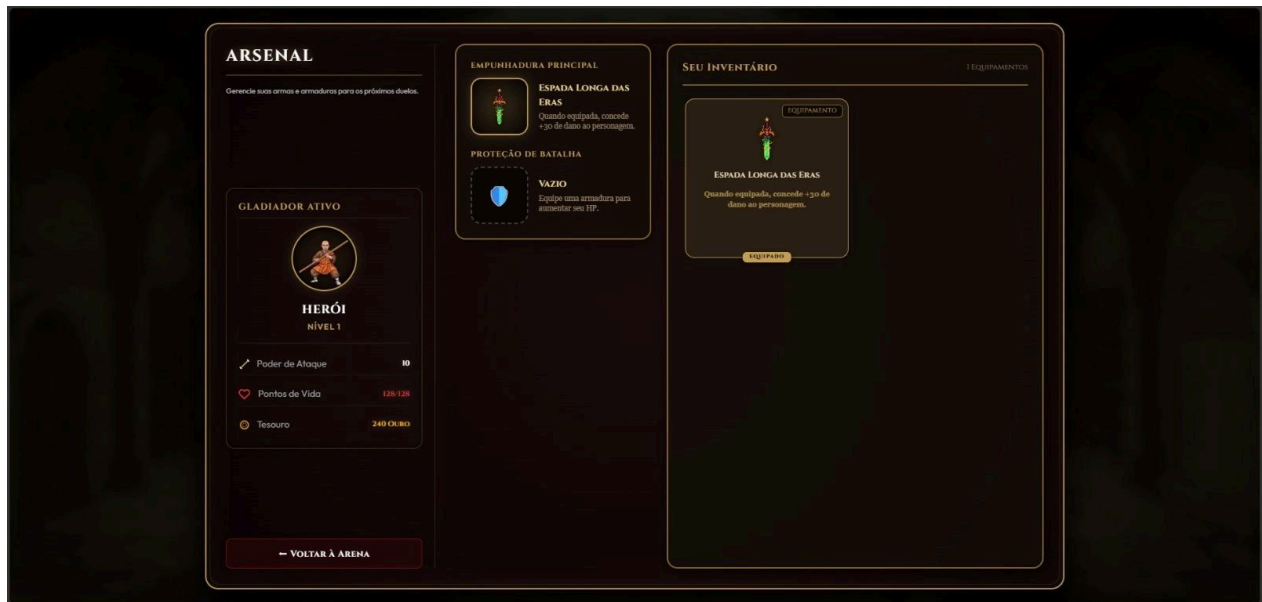


Figura C.16 Arsenal, utilizado para verificar o equipamento e estatísticas básicas do personagem.

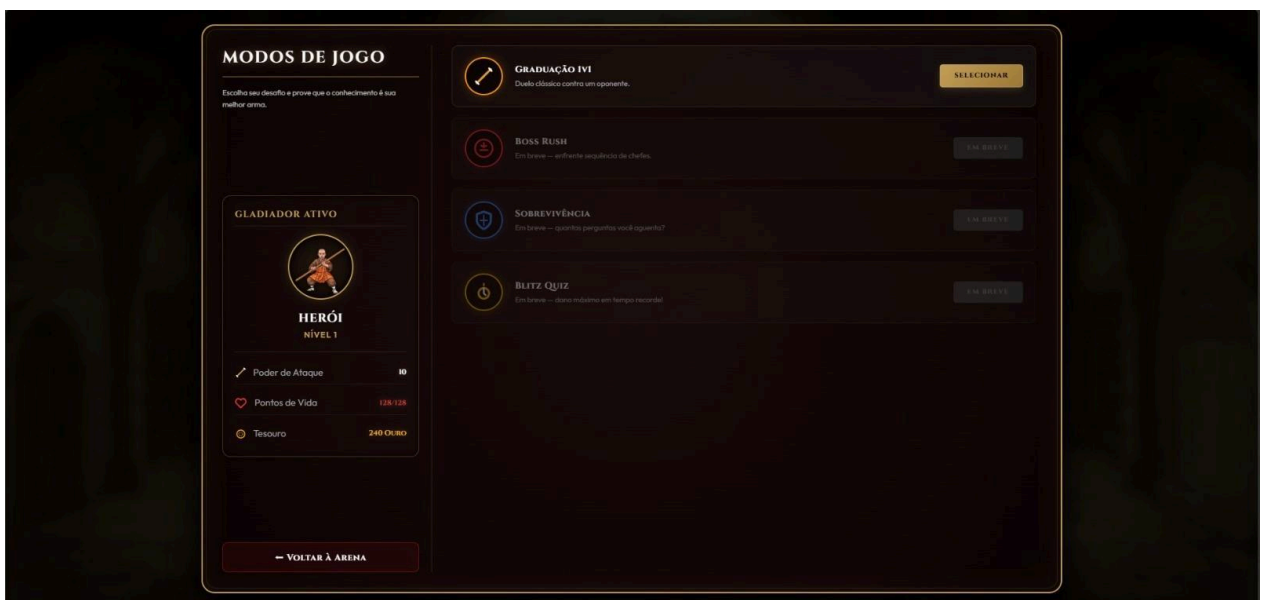


Figura C.17 Modos de Jogo, utilizada para selecionar entre os diferentes modos de jogo disponíveis



Figura C.18 Conquistas, utilizada para verificar o progresso das conquistas.